CINEVERSE

Complete RDBMS Project Evaluation & Viva Voce Preparation Master Guide

---

Course: Relational Database Management Systems Laboratory (CSE)

Institution: Manipal University Jaipur (MUJ)

Database: MySQL 8.0 (InnoDB) — `MovieTheaterDB`

Technology Stack: Node.js, Express, React, Vite, Three.js

Date of Evaluation: October 2026

CINEVERSE — Comprehensive RDBMS Project Evaluation

Academic Lab Project Assessment & Defence Audit Course: Relational Database Management Systems Laboratory (CSE Department) **Institution:** Manipal University Jaipur (MUJ) **Evaluated Database:** MySQL 8.0 (`MovieTheaterDB` — InnoDB Engine) **Evaluated Architecture:** Multi-Tier Architecture (MySQL ↔ Node.js/Express ↔ React Client Applications) **Evaluation Date:** October 2026

Executive Summary & Marking Matrix

This evaluation assesses the CINEVERSE Movie Theatre Booking and Management System strictly against university-level RDBMS criteria. Every score is based on verified codebase artifacts, live schema introspection, SQL query profiling, and transaction traces. No marks are awarded based on user interface claims or unverified documentation.

Evaluation Scorecard (18 Core Areas)

Area #	Evaluated Area	Score (out of 10)	Implementation Status	Key Strength / Primary Risk
1	Problem Statement & Domain Relevance	**9.0 / 10**	Verified Complete	Excellent enterprise domain model with real multi-city scaling.
2	System Architecture & Tier Separation	**8.5 / 10**	Verified Complete	Clean separation of Admin (5173), Customer (5174), and API (5000).
3	Database Design & Relational Modeling	**9.0 / 10**	Verified Complete	15 base tables cleanly separating static venue from dynamic schedules.
4	ER Diagram Correctness & Cardinality	**8.5 / 10**	Verified Complete	All cardinalities, PKs, FKs, and junction tables accurately mapped.
5	Relational Schema & Data Types	**8.5 / 10**	Verified Complete	Strict typing (DECIMAL, TIME, DATE, TIMESTAMP); no loose VARCHARs.
6	Primary, Foreign & Unique Key Design	**9.5 / 10**	Verified Complete	Exceptional composite PKs and multi-column UNIQUE constraints.
7	Relationship Cardinalities (1:1, 1:N, M:N)	**9.0 / 10**	Verified Complete	Perfect junction tables for M:N relationships; explicit 1:1 constraints.
8	Normalization & Functional Dependencies	**8.5 / 10**	Verified Complete	Strict 3NF across all core tables; zero multi-valued dependencies.

Area #	Evaluated Area	Score (out of 10)	Implementation Status	Key Strength / Primary Risk
9	Referential Integrity & Constraints	**9.5 / 10**	Verified Complete	22 CHECK constraints; explicit ON DELETE CASCADE and RESTRICT rules.
10	SQL Query Quality & Efficiency	**8.5 / 10**	Verified Complete	Multi-table INNER/LEFT JOINs, aggregate rollups, parameterized statements.
11	CRUD Operations Implementation	**8.5 / 10**	Verified Complete	Full CRUD on movies, showtimes, venues, bookings, and concessions.
12	Transaction Management & ACID Properties	**8.0 / 10**	Partial Implementation	Multi-statement ACID transaction in booking; missing lock on seat check.
13	Backend Database Driver & Integration	**8.5 / 10**	Verified Complete	Connection pooling (`mysql2/promise`), keepalive, prepared statements.
14	Admin & Customer Frontend Integration	**8.5 / 10**	Verified Complete	Real-time synchronization of bookings, seat maps, and catalog changes.
15	Booking & Seat Availability Consistency	**7.5 / 10**	Partial Implementation	Dynamic availability calculated from DB; concurrency race condition risk.
16	Payment & Financial Settlement Handling	**7.0 / 10**	Verified Simulation	Simulated ledger with 1:1 FK; lacks live external payment gateway.
17	Testing, Validation & Edge Cases	**8.0 / 10**	Verified Complete	37 integration tests pass 100%; lacks unit tests with mock DB fixtures.
18	Documentation, Report & Demo Readiness	**9.0 / 10**	Verified Complete	29-page IEEE/academic PDF report; running dev servers; fast response.
--	**Overall Evaluated Score**	**8.56 / 10**	**Grade: A (Distinction)**	**Highly Defensible University RDBMS Lab Project**

Detailed Evaluation by Area

1. Problem Statement and Objectives

Score: *9.0 / 10* **What is Implemented:** CINEVERSE solves the challenge of centralizing multiplex theater management, scheduling conflicting screenings across physical auditoriums, calculating live seat availability without denormalization, and reconciling ticket and snack revenue. ***Evidence:***
`database/01_create_database.sql` *to* `database/11_add_movie_trailer_and_hero_settings.sql`
`backend/src/controllers/showtimeController.js` *(Lines 176–260: scheduling conflict engine)*
`backend/src/controllers/bookingController.js` *(Lines 136–350: atomic multi-seat booking)*

Design Strengths: Does not treat cinema booking as a toy single-theater model. Accurately maps 8 multiplexes across 5 Indian cities (Bengaluru, Mumbai, Delhi, Hyderabad, Chennai) with 32 auditoriums and 4,544 physical seats. ***Potential Weaknesses:*** *Does not incorporate dynamic pricing algorithms based on time-until-showtime or historical demand, though tiered base pricing by seat category (* `VIP`, `Premium`, `Standard`*) is fully operational.*
Examiner Challenge: *"Why build a bespoke system when commercial SaaS platforms like BookMyShow exist?"*
Defense: Commercial aggregators charge heavy convenience fees and lock theaters out of their raw operational data. CINEVERSE gives theater circuits internal control over auditorium hardware mapping, physical inventory, turnover buffers, and concession margins.

Concrete Improvement: *Implement a loyalty point redemption engine during checkout to complete the customer retention loop.*

---

2. System Architecture

Score: 8.5 / 10 ***What is Implemented:*** *Three-tier client-server architecture with completely decoupled presentation, application, and persistence layers.* **Evidence:**
Presentation (Admin): `frontend-admin/` running on Vite / React 18 at `http://localhost:5173`

Presentation (Customer): `frontend-customer/` running on Vite / React 18 at `http://localhost:5174`

Application Layer: `backend/src/server.js` running Node.js v20 / Express 4.19 at `http://localhost:5000`

Database Layer: MySQL 8.0 `MovieTheaterDB` on port 3306 managed by InnoDB engine

Design Strengths: *Complete separation between the public consumer browsing portal and the administrative back-office control room. The frontends communicate exclusively via JSON over REST; no direct database credentials or queries reside in client code.* **Potential Weaknesses:** The backend serves as a monolithic REST API. While ideal for a laboratory project, a distributed enterprise deployment would decouple the booking transaction engine from catalog reporting. ***Examiner Challenge:*** "Why run two separate React frontends on different ports instead of role-based route guards in a single app?"
Defense: *Strict security boundary separation. The admin bundle includes staff authentication, inventory control, and scheduling collision interfaces that should never be sent over the wire to retail customer browsers.*

Concrete Improvement: Add an API Gateway (or Nginx reverse proxy) to expose a unified domain on port 80/443 with path-based routing (`/api` , `/admin` , `/`).

---

3. Database Design

Score: *9.0 / 10* **What is Implemented:** 15 base relational tables organized into three distinct architectural domains:
Infrastructure Domain: `THEATER` , `SCREEN` , `SEAT` , `ADMIN_STAFF`

Theatrical Programming Domain: `MOVIE` , `SHOWTIME`

Transactional Commercial Domain: `CUSTOMER` , `ACCOUNT` , `LOYALTY_ACCOUNT` , `BOOKING` , `BOOKED_SEAT` , `PAYMENT` , `SNACK` , `SNACK_ORDER` , `SNACK_ORDER_ITEM`

Evidence:
Live table count verified via `SHOW FULL TABLES WHERE Table_type = 'BASE TABLE'`

Total physical database rows: 5,699 records across 15 tables.

Design Strengths: Clean separation between static venue assets and dynamic show schedules. A physical seat is modeled once in `SEAT` and never modified or flagged with a text state during booking; availability is determined

relationally through `BOOKED_SEAT`. ***Potential Weaknesses:*** *Absence of database views (`VIEW`) or stored procedures (`PROCEDURE`), as business logic is currently encapsulated in backend controllers.* **Examiner Challenge:** *"Why isn't seat availability stored as a boolean column `is_available` directly inside the `SEAT` table?"*
Defense: Storing `is_available` on `SEAT` violates relational principles. A physical seat is available or booked relative to a specific *showtime*, not in perpetuity. Storing status in `SEAT` would prevent scheduling concurrent or future screenings in that auditorium.

Concrete Improvement: *Create a MySQL View `vw_live_seat_availability` joining `SHOWTIME`, `SEAT`, and `BOOKED_SEAT`.*

---

4. ER Diagram Correctness

Score: 8.5 / 10 ***What is Implemented:*** *Entity-Relationship diagram completely captures all 15 entities, primary keys, foreign key lines, and Crow's Foot cardinality notations.* **Evidence:**
`CINEVERSE_ER_Diagram.mmd` (Mermaid code)

`report_assets/cineverse_er_diagram.svg` and `report_assets/fig_3_1_er_diagram.png`

Verified relationship lines: 15 explicit foreign key relationships.

Design Strengths: *Accurate modeling of associative entities (junction tables) `BOOKED_SEAT` and `SNACK_ORDER_ITEM` resolving Many-to-Many relationships.* **Potential Weaknesses:** The Mermaid ER diagram models `CUSTOMER` ↔ `ACCOUNT` as a 1:1 relationship, but the ER notation in standard Chen notation could distinguish the total vs partial participation. ***Examiner Challenge:*** "Which entities in your ER diagram are strong entities and which are weak entities?"
Defense: *`THEATER`, `MOVIE`, `CUSTOMER`, and `SNACK` are strong entities (exist independently). `SCREEN` and `SEAT` can be considered structurally dependent on `THEATER` and `SCREEN` respectively (existence dependency). `BOOKED_SEAT` is an associative entity whose identity depends on `BOOKING` and `SEAT`.*

Concrete Improvement: Add Chen ER diagram notation side-by-side with Crow's Foot to demonstrate dual theoretical mastery.

---

5. Relational Schema and Data Types

Score: *8.5 / 10* **What is Implemented:** Explicit data typing utilizing appropriate relational primitives:
Monetary values: `DECIMAL(10,2)` for zero round-off error

Time tracking: `TIME` for start/end, `DATE` for show date, `TIMESTAMP` with `CURRENT_TIMESTAMP` for audit logging

Text fields: Bounded `VARCHAR(100)` to `VARCHAR(150)`; no unbounded `TEXT` fields

Numerical counts: `INT` with `capacity > 0` and `duration > 0`

Evidence:
Verified via `INFORMATION_SCHEMA.COLUMNS` across all 15 tables.

Design Strengths: Zero use of floating-point types (`FLOAT` or `DOUBLE`) for financial data, preventing binary floating-point rounding anomalies during revenue aggregation. ***Potential Weaknesses:*** *Movie ratings use `DECIMAL(3,1)` allowing values up to 99.9 if unchecked, though constrained to ≤ 10.0 by check constraint*

`chk_movie_rating`. **Examiner Challenge:** *"Why use `DECIMAL(10,2)` instead of `FLOAT` or `DOUBLE` for ticket prices?"*
Defense: `FLOAT` and `DOUBLE` use IEEE 754 binary floating-point representation, which cannot precisely represent fractional decimal powers (e.g., 0.10). In financial ledgers, accumulating rounding discrepancies violates accounting standards. `DECIMAL` stores numbers as fixed-point exact integers.

Concrete Improvement: *Standardize currency ISO codes by adding a `currency_code CHAR(3) DEFAULT 'INR'` column to `THEATER`.*

---

6. Primary and Foreign Keys

Score: 9.5 / 10 ***What is Implemented:*** *Every table possesses a declared Primary Key. All 15 foreign key constraints are enforced by the InnoDB engine with explicit constraint names.* **Evidence:**
Verified via `INFORMATION_SCHEMA.KEY_COLUMN_USAGE`

Composite Primary Keys:

`BOOKED_SEAT`: `PRIMARY KEY (booking_id, seat_id)` `SNACK_ORDER_ITEM`: `PRIMARY KEY (snack_order_id, snack_id)`
Composite Unique Constraints:

`SCREEN`: `UNIQUE KEY uq_theater_screen (theater_id, screen_number)` `SEAT`: `UNIQUE KEY uq_screen_seat (screen_id, seat_number)` ***Design Strengths:*** *Composite unique keys prevent real-world duplication: Screen 1 cannot exist twice within the same theater; Seat A1 cannot exist twice within the same auditorium screen.* **Potential Weaknesses:** None found. Primary and foreign key architecture is exemplary.
Examiner Challenge: "What is the difference between a Primary Key and a Unique Constraint?"
Defense: *A table can have only one Primary Key, which strictly forbids `NULL` values. A table can possess multiple `UNIQUE` constraints. In CINEVERSE, `seat_id` is the surrogate PK, while `(screen_id, seat_number)` is an alternate/candidate key enforced via a `UNIQUE` constraint.*

Concrete Improvement: None required.

---

7. Relationship Cardinalities

Score: *9.0 / 10* **What is Implemented:**
One-to-One (1:1): `CUSTOMER` ↔ `ACCOUNT` (enforced by `UNIQUE (customer_id)` on `ACCOUNT`), `CUSTOMER` ↔ `LOYALTY_ACCOUNT` (enforced by `UNIQUE (customer_id)` on `LOYALTY_ACCOUNT`), `BOOKING` ↔ `PAYMENT` (enforced by `UNIQUE (booking_id)` on `PAYMENT`).

One-to-Many (1:N): `THEATER` → `SCREEN`, `SCREEN` → `SEAT`, `MOVIE` → `SHOWTIME`, `SCREEN` → `SHOWTIME`, `CUSTOMER` → `BOOKING`, `SHOWTIME` → `BOOKING`, `BOOKING` → `SNACK_ORDER`.

Many-to-Many (M:N): `BOOKING` ↔ `SEAT` (via `BOOKED_SEAT`), `SNACK_ORDER` ↔ `SNACK` (via `SNACK_ORDER_ITEM`).

Evidence:
Live constraint definitions inspected in `INFORMATION_SCHEMA.STATISTICS` and `KEY_COLUMN_USAGE`.

Design Strengths: Realizes 1:1 relationships using unique foreign keys rather than merging attributes into a bloated mega-table, preserving separation of concerns (patron demographics vs salted password credentials).
Potential Weaknesses: *None. Cardinalities faithfully mirror the business rules of cinema multiplex operations.*

Examiner Challenge: *"Why is the relationship between BOOKING and SEAT Many-to-Many if a seat can only be booked once for a showtime?"*
Defense: A physical seat (e.g., Seat 101) can be booked across *hundreds* of different showtimes over the year. Simultaneously, a single booking reservation can encompass *multiple* seats (e.g., a family booking 4 seats). Therefore, the relationship between the `BOOKING` entity and the physical `SEAT` entity is Many-to-Many ($M:N$), necessitating the `BOOKED_SEAT` junction table.

Concrete Improvement: *None required.*

---

8. Normalization and Functional Dependencies

Score: 8.5 / 10 ***What is Implemented:*** *The schema satisfies 1NF, 2NF, and 3NF across all 15 tables.* **Evidence:**
1NF: All attributes are atomic (no comma-separated seats or array types).

2NF: In tables with composite keys (`BOOKED_SEAT` and `SNACK_ORDER_ITEM`), non-key attributes (`ticket_price` , `quantity` , `unit_price`) depend on the *entire* composite key, not a proper subset.

3NF: No non-key attribute is transitively dependent on the primary key via another non-key attribute. For example, `SCREEN` contains `theater_id` , while theater address and city remain strictly in `THEATER` .

Design Strengths: *High structural integrity. Modifying a theater's street address requires updating exactly one row in `THEATER` , eliminating update anomalies across screens and showtimes.* **Potential Weaknesses:** Controlled denormalization: `BOOKED_SEAT` stores `ticket_price` at the moment of reservation. This is an intentional historical financial snapshot, not an unnormalized redundancy anomaly. ***Examiner Challenge:*** "Does `BOOKED_SEAT.ticket_price` violate 3NF since base price is already stored in `SHOWTIME` ?"
Defense: *No. In transactional accounting, recording the executed ticket price is mandatory. If ticket base prices in `SHOWTIME` or tier multipliers change in the future, historical accounting records must reflect the exact price paid by the customer at the time of purchase. Storing this value preserves immutable financial history.*

Concrete Improvement: Formally document this design decision in the project report as intentional temporal snapshot denormalization.

---

9. Referential Integrity and Constraints

Score: *9.5 / 10* **What is Implemented:** Comprehensive integrity protection at both schema and database engine levels:
22 Active CHECK Constraints: Validating email formatting, rating bounds ($0.0 \le rating \le 10.0$), positive quantities, and allowed ENUM-like sets (`membership_level` , `payment_method` , `booking_status` , `seat_type`).

Cascading Deletions: `ON DELETE CASCADE` on `account` , `loyalty_account` , `screen` , `seat` , `booked_seat` , `payment` , `snack_order` , `snack_order_item` .

Restrictive Deletions: `ON DELETE RESTRICT` on `booking -> customer` , `booking -> showtime` , `showtime -> movie` , `showtime -> screen` , preventing catastrophic accidental deletion of commercial records.

Evidence:
Audited directly via `INFORMATION_SCHEMA.CHECK_CONSTRAINTS` and `INFORMATION_SCHEMA.REFERENTIAL_CONSTRAINTS` .

Design Strengths: Industry-grade referential integrity. Deleting a Movie that has scheduled showtimes or bookings is rejected outright by the MySQL engine. ***Potential Weaknesses:*** *None found.* **Examiner Challenge:** *"What happens if an admin tries to delete a Movie that has active ticket bookings?"*
Defense: The operation fails at two levels: First, `backend/src/controllers/movieController.js` performs a pre-flight query checking `SELECT show_id FROM SHOWTIME WHERE movie_id = ?` and returns HTTP 409 Conflict. Second, even if bypassed, MySQL's foreign key constraint `fk_showtime_movie` enforces `ON DELETE RESTRICT` and aborts the query with Error 1451.

Concrete Improvement: *None required.*

---

10. SQL Query Quality

Score: 8.5 / 10 ***What is Implemented:*** *SQL queries demonstrate high proficiency in relational operations: Multi-table INNER JOINs spanning up to 6 tables (`BOOKING`, `CUSTOMER`, `SHOWTIME`, `MOVIE`, `SCREEN`, `THEATER`)*

Aggregations: `SUM`, `AVG`, `COUNT(DISTINCT ...)`, `COALESCE`

Grouping and Filtering: `GROUP BY` *with* `HAVING`, `WHERE 1=1` *dynamic filtering*

String manipulation: `GROUP_CONCAT(st.seat_number ORDER BY st.seat_number SEPARATOR ', ')`

100% Parameterized query execution using `?` *placeholders preventing SQL injection.*

Evidence:

`backend/src/controllers/bookingController.js` (Lines 9–51, 65–125)

`backend/src/controllers/reportController.js` (Lines 9–40: revenue rollups)

Design Strengths: *Prepared statements (`pool.execute()`) leverage the MySQL binary protocol for query plan caching and injection immunity.* **Potential Weaknesses:** Subqueries could be leveraged more heavily in place of multiple sequential round trips in certain controller functions. ***Examiner Challenge:*** "What is the performance drawback of `GROUP_CONCAT` in your `getAllBookings` query?"
Defense: `GROUP_CONCAT` *is subject to MySQL's* `group_concat_max_len` *system variable (default 1024 bytes). For bookings with an unusually high number of seats, seat strings exceeding this limit would be silently truncated. In production, this can be mitigated by adjusting* `group_concat_max_len` *or querying seat allocations in a dedicated child query.*

Concrete Improvement: Add query indexing on `SHOWTIME(show_date, start_time)` to optimize screening range lookups.

---

11. CRUD Implementation

Score: *8.5 / 10* **What is Implemented:** Full Create, Read, Update, Delete capabilities exposed through REST endpoints:
Movies: Create (`POST /api/admin/movies`), Read (`GET /api/movies`), Update (`PUT /api/admin/movies/:id`), Delete (`DELETE /api/admin/movies/:id`)

Showtimes: Create with collision check (`POST /api/admin/showtimes`), Read (`GET /api/showtimes`), Delete (`DELETE /api/admin/showtimes/:id`)

Bookings: Create (`POST /api/bookings`), Read (`GET /api/bookings`), Cancel/Update (`PUT /api/bookings/:id/cancel`)

Evidence:

`backend/src/routes/api.js`

`backend/src/controllers/movieController.js`

`backend/src/controllers/showtimeController.js`

`backend/src/controllers/bookingController.js`

Design Strengths: Input sanitization and conflict validation precede all DML execution. ***Potential Weaknesses:*** *Customer profile updates are not fully exposed in the retail frontend (customers cannot edit phone/email in the UI).*
Examiner Challenge: *"How does your system handle booking deletion?"*
Defense: We do not physically execute `DELETE FROM BOOKING`. Deleting financial transactions destroys audit trails. Instead, we perform a logical/soft cancellation: `UPDATE BOOKING SET booking_status = 'CANCELLED'` and `UPDATE PAYMENT SET payment_status = 'REFUNDED'`. This immediately releases the seats for re-booking while preserving financial history.

Concrete Improvement: *Implement customer profile self-service editing endpoints.*

---

12. Transaction Management and Concurrency

Score: 8.0 / 10 ***What is Implemented:*** *Multi-statement ACID transactions implemented via `conn.beginTransaction()`, `conn.commit()`, and `conn.rollback()` in `bookingController.js` and `showtimeController.js`.* **Evidence:**
`backend/src/controllers/bookingController.js` (Lines 154–351: `createBooking`)

`backend/src/controllers/bookingController.js` (Lines 363–407: `cancelBooking`)

Design Strengths: *Atomicity is strictly respected. If snack inventory deduction fails after seat reservation, or payment fails, the entire transaction rolls back completely, leaving zero orphan rows.* **Potential Weaknesses:** Concurrency race condition on seat selection! Step 5 of `createBooking` queries `BOOKED_SEAT` without an explicit row lock (`FOR UPDATE`) or serializable isolation. Two simultaneous requests checking the same seat could both see it as available before either commits. ***Examiner Challenge:*** "What happens if two users click 'Confirm Booking' for the exact same seat at the exact same millisecond?"
Defense (Honest & Defensible): *In the current implementation, because the seat conflict query in Step 5 does not execute `SELECT ... FOR UPDATE`, and MySQL's default isolation level is `REPEATABLE READ`, both transactions could pass the conflict check and attempt insertion. While the application checks conflicts, under high concurrency a race condition is theoretically possible. To make this 100% concurrency-proof at the database engine level, we must either add `FOR UPDATE` to the seat validation query, or enforce an engine-level unique key constraint on `(show_id, seat_id)`.*

Concrete Improvement: Add `FOR UPDATE` to the seat conflict check query or create a reservation lock table.

---

13. Backend Database Driver and Integration

Score: *8.5 / 10* **What is Implemented:** Connection pooling implemented via `mysql2/promise` with configuration parameters loaded from environment variables (`.env`). ***Evidence:***
`backend/src/config/db.js`

Connection Pool Config: `connectionLimit: 10`, `queueLimit: 0`, `waitForConnections: true`, `enableKeepAlive: true`.

Design Strengths: Asynchronous non-blocking I/O using native ES6 `async/await`. Connection pooling prevents socket exhaustion under bursts of HTTP requests. ***Potential Weaknesses:*** *Connection pool size is fixed at 10; could be tuned dynamically based on production CPU cores.* **Examiner Challenge:** *"Why use connection pooling instead of opening a new `mysql.createConnection()` for each incoming HTTP request?"*
Defense: Establishing a new TCP handshake and MySQL authentication session takes 20–50ms per request. Under 100 concurrent requests, creating individual connections exhausts server sockets and crashes MySQL. A connection pool keeps a pool of authenticated, ready-to-use connections alive, drastically reducing query latency to sub-2ms.

Concrete Improvement: *Implement pool health monitoring and alert logging if queue wait time exceeds 200ms.*

---

14. Admin and Customer Frontend Integration

Score: 8.5 / 10 ***What is Implemented:*** *Two dedicated Single-Page Applications (SPAs) built with React 18, utilizing modern hooks (`useState`, `useEffect`, `useMemo`, `useCallback`) and Axios API service abstraction.*
Evidence:
`frontend-admin/src/views/` (`MoviesView.jsx`, `ShowtimesView.jsx`, `TheatersView.jsx`, `HeroTrailersView.jsx`)

`frontend-customer/src/views/` (`HomeView.jsx`, `TheatresView.jsx`, `BookingFlowView.jsx`, `MyBookingsView.jsx`)

Design Strengths: *Rich interactive experiences: dynamic 3D WebGL canvas (`Three.js`), interactive theater seat picker with tier colors, and instant feedback.* **Potential Weaknesses:** State management uses prop-drilling and local component state rather than a global Redux/Zustand store. ***Examiner Challenge:*** "How does the customer seat map update when an admin schedules a new showtime or cancels a screening?"
Defense: *The customer frontend fetches showtime seats dynamically upon navigating to `/booking-flow` by issuing `GET /api/showtimes/:id/seats`. This hits the database directly, returning the up-to-the-second seat map.*

Concrete Improvement: Introduce WebSockets (`Socket.io`) for real-time live seat holding across multiple active user browsers.

---

15. Booking and Seat Availability Consistency

Score: *7.5 / 10* **What is Implemented:** Dynamic seat calculation:
`total_seats = seats.length`

`booked_seats = bookedSeatIds.size` (from active bookings `booking_status != 'CANCELLED'`)

`available_seats = total_seats - booked_seats`

Evidence:

`backend/src/controllers/showtimeController.js` *(Lines 48–94:* `getShowtimeSeats`*)*

Design Strengths: Does not rely on stale counters or cached seat states. Seat availability is dynamically derived from the relational ledger. ***Potential Weaknesses:*** *No temporary "Seat Holding / Cart Reservation" mechanism. If a user selects a seat, it is not held while they fill in payment details; it is only secured upon final booking submission.*
Examiner Challenge: *"What prevents a user from holding 10 seats indefinitely without paying?"*
Defense: In CINEVERSE, seats are not locked during selection; they are committed to the database only when the transaction succeeds. This guarantees that seats are never locked up by abandoned carts. However, in high-volume

ticketing, an industry improvement is to implement an ephemeral 5-minute Redis TTL seat lock.

Concrete Improvement: *Implement an expiring seat-reservation token with a 5-minute timeout.*

---

16. Payment Data Handling

Score: 7.0 / 10 ***What is Implemented:*** *Simulated payment settlement engine. When a booking request is submitted, a* `PAYMENT` *row is inserted with* `payment_status = 'COMPLETED'`, `payment_method` *(*`UPI`, `Credit Card`, `Debit Card`, `Net Banking`, `Cash`*), and amount matching the booking total.* **Evidence:** `backend/src/controllers/bookingController.js` (Lines 305–311)

`database/02_create_tables.sql` (`PAYMENT` table definition with `uq_payment_booking`)

Design Strengths: *Relational integrity is strictly enforced via* `uq_payment_booking UNIQUE (booking_id)` *ensuring a 1:1 relationship between ticket invoices and payment receipts.* **Potential Weaknesses:** Does not communicate with a real banking payment gateway (e.g., Razorpay or Stripe) over webhook callbacks. ***Examiner Challenge:*** "Is your payment gateway real or simulated? What PCI-DSS risks would exist if this went live?" ***Defense:*** *It is an internal academic simulation. If deployed commercially, storing raw credit card details or handling payments without PCI-DSS compliance is illegal. A production deployment would use hosted checkout sessions (e.g., Stripe/Razorpay) where credit card numbers never touch our servers, and our database only stores the gateway's tokenized transaction ID.*

Concrete Improvement: Integrate Razorpay Test Sandbox with webhook-verified signature validation.

---

17. Testing and Validation

Score: *8.0 / 10* **What is Implemented:** 37 automated integration tests verifying database release status queries, conflict-free scheduling, admin trailer CRUD, URL normalization, and seat isolation. 100% pass rate. ***Evidence:*** `REPORT_VERIFICATION.md` *(Section 4: Test Execution)*

`scratch/comprehensive_test.js` *(37 passed, 0 failed)*

Production build verifications: `npm run build` *succeeds with 0 errors across both React frontends.*

Design Strengths: Tests are executed against live running servers and live MySQL database, not mocked memory stubs. ***Potential Weaknesses:*** *Lacks an automated continuous integration test runner (e.g., Jest or Vitest) configured in* `backend/package.json` *scripts.* **Examiner Challenge:** *"How did you verify that your scheduling collision detection works for adjacent and overlapping time slots?"*
Defense: We validated the mathematical interval condition $(s_{start} < end_{calc} \land s_{end} > start_{form})$ against multiple test scenarios: identical start time, screening starting inside an existing screening, screening ending after another starts, and sequential screening with insufficient turnaround buffer (< 15 mins). All invalid cases were rejected with HTTP 409 Conflict.

Concrete Improvement: *Add Jest test scripts to* `backend/package.json` *with code coverage reports.*

---

18. Documentation, Report and Demo Readiness

Score: 9.0 / 10 ***What is Implemented:*** *Complete academic deliverables package:*
29-page publication-quality PDF report (`CINEVERSE_RDBMS_Project_Report.pdf`*) matching Manipal University Jaipur formatting.*

Fully editable Word document (`CINEVERSE_RDBMS_Project_Report.docx`).

High-resolution ER diagram vector SVG and raster PNG (`report_assets/`).

Live servers running locally on ports 5000, 5173, and 5174 ready for immediate inspection.

Evidence:
Verified deliverables on disk in `d:\Desktop\Projects\RDBMS\`.

Design Strengths: *Report contains zero placeholder text in technical chapters, verified table row counts, and authentic screenshots.* **Potential Weaknesses:** Cover page requires student registration details to be filled in. ***Examiner Challenge:*** "Can you show me the exact database rows that support your admin dashboard KPIs?" ***Defense:*** *Yes. The admin dashboard queries* `SELECT COUNT( ) FROM MOVIE` (13), `SHOWTIME` (1008), `BOOKING` (19), and `CUSTOMER` (10). We can verify these numbers live in MySQL CLI during the viva.

Concrete Improvement: *Fill in student personal details on the cover page.*

Examiner Challenges & Strategic Improvement Plan

Top 5 Examiner Vulnerabilities to Defend

Seat Booking Concurrency (Race Condition):

Risk: *No* `SELECT ... FOR UPDATE` *on* `BOOKED_SEAT` *during Step 5 of* `createBooking`.

Defense: *Acknowledge the theoretical limitation under high concurrent volume. Explain that MySQL's InnoDB transactions and the application conflict check handle sequential and isolated requests cleanly, but an enterprise deployment requires* `FOR UPDATE` *row locking or an engine-level unique key constraint on* `(show_id, seat_id)`.

Simulated Payment Gateway:

Risk: *No real third-party payment gateway integration.*

Defense: *Clarify that the project's primary scope is RDBMS schema design and transactional data reconciliation. Financial reconciliation is accurately modeled via* `PAYMENT` *with 1:1 constraints and* `COMPLETED`/`REFUNDED` *status transitions.*

Absence of Database Triggers and Stored Procedures:

Risk: *Examiner expects stored procedures or triggers in an RDBMS lab.*

Defense: *Explain that modern enterprise software engineering often places business validation in the application service layer for portability, unit testability, and horizontal scaling. However, demonstrate your mastery of SQL by offering the exact DDL/SQL code for a stored procedure or trigger (provided in Section E & J of the Viva Pack).*

Movie Catalog Fallback Settings:

Risk: `trailer_url` *and release dates are supplemented by* `backend/data/hero_settings.json` *when columns are absent in the initial base migration.*

Defense: *Migration scripts* `10_add_movie_release_management.sql` *and* `11_add_movie_trailer_and_hero_settings.sql` *have been prepared to add these columns natively. The JSON fallback ensures backwards compatibility without executing destructive unverified migrations.*

No Ephemeral Seat Holding (Cart Lock):

Risk: *Seat availability is instantaneous rather than held for 5 minutes during checkout.*

Defense:* Explain that in an ACID RDBMS model, long-lived locks in the database reduce transaction throughput. Commercial systems use Redis in-memory caches for 5-minute cart locks, which sits outside the scope of a core relational database evaluation.

CINEVERSE — Comprehensive Viva Voce Question Bank

Official Examination Preparation Pack Course: Relational Database Management Systems Laboratory (CSE Department) **Institution:** Manipal University Jaipur (MUJ) **Target Project:** CINEVERSE Movie Theatre Booking & Management System **Database:** MySQL 8.0 (`MovieTheaterDB`) | **Stack:** Node.js, Express, React, Vite

---

Guide for the Viva Voce

How to Speak: *Deliver your answers with confidence. Keep the first sentence direct and punchy, followed by the specific database concept (table name, key, constraint, or SQL clause).* **Grounding in Code:** Always reference the actual project tables (`SHOWTIME` , `BOOKED_SEAT` , `PAYMENT` , etc.) rather than speaking abstractly. ***Separation of Facts:*** *Distinguish between what is implemented in the database vs. what is handled by the application logic.*

Section A: Project Overview & Motivation

Q1. What is CINEVERSE, and what primary problem does it solve?

Spoken Answer: *"CINEVERSE is an enterprise-grade multiplex theater booking and management system. It solves the operational complexity of managing multi-city cinema complexes, scheduling collision-free screenings across auditoriums, calculating live seat availability without data redundancy, and executing atomic ticket and concession purchases."*

Q2. Why did you choose a cinema ticketing system for an RDBMS lab project?

Spoken Answer: *"Because cinema ticketing exhibits classic transactional relational challenges: Many-to-Many relationships between bookings and physical seats, strict scheduling constraints to prevent auditorium overlap, inventory control for concessions, and the absolute requirement for ACID transactions to prevent double-booking and financial discrepancies."*

Q3. What are the key functional modules in CINEVERSE?

Spoken Answer: *"The project comprises four core modules:*
Venue Infrastructure Module: *Managing theaters, screens, and tiered physical seating.*

Theatrical Programming Module: *Movie cataloging and showtime scheduling with automated turnaround buffers.*

Customer Ticketing & Booking Engine: *Real-time seat map selection, concession add-ons, and checkout.*

Administrative Back-Office Control Room: *Staff authentication, catalog CRUD, scheduling conflict prevention, and revenue reporting."*

Q4. Who are the primary target users of the system?

Spoken Answer: *"There are two distinct user roles:*
Retail Moviegoers (Customers): *Who browse movies, select theaters by city, pick seats from an interactive auditorium layout, and receive confirmed booking invoices.*

Cinema Operations Staff & Managers (Admins): *Who manage the movie catalog, configure screens, schedule showtimes, and audit ticket and snack revenues."*

Q5. Why is a Relational Database Management System (RDBMS) necessary for this project instead of a simple file system or spreadsheets?

Spoken Answer: *"A file system or spreadsheet lacks concurrent multi-user access control, cannot enforce referential integrity across related tables, does not support ACID transactions, and suffers from data redundancy and update anomalies. An RDBMS enforces schema constraints, indexes data for fast retrieval, and guarantees atomicity during multi-seat reservations."*

Q6. What is the scope of CINEVERSE, and what are its current boundaries?

Spoken Answer: *"The current scope encompasses 8 multiplex venues across 5 Indian cities, 32 auditorium screens, 4,544 physical seats, and over 1,000 scheduled screenings. Its current boundaries are that payment processing is an internal ledger simulation rather than a live banking gateway, and seat holds are instantaneous upon transaction commitment rather than an ephemeral 5-minute cart timer."*

Q7. What was your individual contribution to the project?

Spoken Answer: *"I designed the 3NF relational schema in MySQL, authored the DDL scripts with 22 check constraints and foreign keys, developed the Node.js/Express backend API with connection pooling and ACID transaction rollback, created the scheduling conflict algorithm, and built the React customer and admin web applications."*

Q8. What was the most technically challenging database problem you solved?

Spoken Answer: *"Designing the seat reservation model. Instead of denormalizing an `is_available` flag onto the physical `SEAT` table—which would prevent scheduling future screenings—I implemented an associative junction table `BOOKED_SEAT` linked to `BOOKING` and `SHOWTIME`. Live seat availability is calculated dynamically by relationally subtracting booked seats from total auditorium seats for that specific screening."*

Q9. How does CINEVERSE handle multi-city multiplex operations?

Spoken Answer: *"Through the hierarchical relationship: `THEATER` → `SCREEN` → `SEAT`. The `THEATER` table stores venue city and address. Customer queries filter screens and showtimes based on the chosen city, allowing patrons in Bengaluru, Mumbai, or Delhi to view only locally scheduled screenings."*

Q10. How does the system handle movies that have no scheduled showtimes?

Spoken Answer: *"A movie can exist in the `MOVIE` catalog independently of `SHOWTIME`. If a movie has zero scheduled showtimes, the customer frontend categorizes it as 'Coming Soon' or 'Showtimes TBA'. It only becomes bookable once an administrator schedules at least one valid showtime in an auditorium."*

Section B: Architecture & Technology Stack

Q11. What is the architecture of the CINEVERSE application?

Spoken Answer: *"CINEVERSE follows a decoupled Three-Tier Architecture:*
Presentation Tier: *Two React 18 single-page applications built with Vite (Admin on port 5173, Customer on port 5174).*

Application Tier: *Node.js v20 with Express 4.19 REST API running on port 5000.*

Data Tier: *MySQL 8.0 running on port 3306 with the InnoDB storage engine."*

Q12. Why did you separate the Admin and Customer frontends into two different applications?

Spoken Answer: *"For security and domain separation. The admin application contains sensitive control room components—such as JWT authentication, scheduling conflict engines, and revenue reporting—that should never be delivered in the JavaScript bundle to public retail users."*

Q13. Why did you choose MySQL over a NoSQL database like MongoDB?

Spoken Answer: *"Cinema ticketing is fundamentally relational and requires strict ACID compliance. NoSQL document stores permit denormalization and eventual consistency, which creates severe risks of double-booking under concurrent reservations. MySQL provides foreign key referential integrity, multi-table joins, and InnoDB row-level locking to guarantee zero data corruption."*

Q14. How does the Node.js backend connect to the MySQL database?

Spoken Answer: *"We use the* `mysql2/promise` *driver with connection pooling. In* `backend/src/config/db.js`*, we initialize a pool with 10 connections, keepalive enabled, and queue limit zero. Queries are executed asynchronously using* `async/await` *and parameterized prepared statements."*

Q15. What is the role of Express.js in this architecture?

Spoken Answer: *"Express serves as the HTTP routing and middleware framework. It parses incoming JSON request payloads, manages CORS headers between ports, handles JWT authorization headers via middleware, routes requests to dedicated controllers, and catches errors in a centralized error-handling middleware."*

Q16. Can you trace what happens from the moment a user clicks 'Book Seats' to the database?

Spoken Answer: *"1. The React frontend sends an HTTP* `POST` *request with a JSON body (* `customer_id`*,* `show_id`*,* `seat_ids`*,* `payment_method`*,* `snacks`*) to* `/api/bookings`*.*
Express routes this to `createBooking` *in* `bookingController.js`*.*

The controller obtains a dedicated connection from the pool and calls `conn.beginTransaction()`*.*

It validates the customer and showtime, checks that seats belong to the scheduled screen, and verifies no active booking already claims those seats.

It inserts into `BOOKING`*, loops over seats inserting into* `BOOKED_SEAT`*, inserts a* `PAYMENT` *record, deducts snack inventory, and calls* `conn.commit()`*.*

It releases the connection back to the pool and returns HTTP 201 Created."

Q17. What is a REST API, and how does CINEVERSE implement it?

Spoken Answer: *"A REST API (Representational State Transfer) uses standard HTTP methods (* `GET`*,* `POST`*,* `PUT`*,* `DELETE`*) to perform operations on URI-identified resources. In CINEVERSE,* `GET /api/movies` *retrieves the catalog,* `POST /api/bookings` *creates a reservation, and* `PUT /api/bookings/:id/cancel` *performs a logical cancellation."*

Q18. How do environment variables protect your database configuration?

Spoken Answer: *"In `backend/.env`, we store `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, and `JWT_SECRET`. We load them using the `dotenv` package. This keeps sensitive database credentials out of version control (`.gitignore`), preventing security leaks."*

Q19. What is Vite, and why did you use it instead of Create React App?

Spoken Answer: *"Vite is a modern frontend build tool that leverages native ES modules in the browser. It provides instantaneous server startup (< 900ms) and lightning-fast Hot Module Replacement (HMR) compared to the deprecated Webpack-based Create React App."*

Q20. What storage engine does MySQL use for CINEVERSE, and why?

Spoken Answer: *"We strictly use InnoDB. InnoDB is MySQL's ACID-compliant storage engine that supports foreign key constraints, row-level locking, and crash recovery. Legacy engines like MyISAM only support table-level locking and do not enforce foreign keys."*

Section C: ER Diagram & Relational Model

Q21. What is an Entity in an ER diagram? Give examples from CINEVERSE.

Spoken Answer: *"An entity is a real-world object or concept that has an independent existence and about which data is stored. In CINEVERSE, `MOVIE`, `THEATER`, `CUSTOMER`, and `SNACK` are strong entities."*

Q22. What is the difference between an Entity and an Attribute?

Spoken Answer: *"An entity is the distinct object itself (e.g., `MOVIE`), whereas an attribute is a characteristic or property that describes that entity (e.g., `title`, `duration`, `genre`, `rating`)."*

Q23. What are Strong and Weak Entities? Are there any weak entities in your schema?

Spoken Answer: *" `THEATER` and `MOVIE` are strong entities because they have independent primary keys. `SCREEN` and `SEAT` exhibit existence dependency—a screen cannot exist without a theater, and a seat cannot exist without an auditorium screen. However, because we assign surrogate primary keys (`screen_id`, `seat_id`), they are implemented as regular relational tables with foreign keys and composite unique constraints."*

Q24. What is a Composite Key? Where is it used in CINEVERSE?

Spoken Answer: *"A composite key is a primary key that consists of two or more columns combined to uniquely identify a record. In CINEVERSE:*
`BOOKED_SEAT` uses `PRIMARY KEY (booking_id, seat_id)`.
`SNACK_ORDER_ITEM` uses `PRIMARY KEY (snack_order_id, snack_id)`."

Q25. What is a Candidate Key? Give an example from your tables.

Spoken Answer: *"A candidate key is any minimal set of attributes that can uniquely identify a tuple in a relation. In our `CUSTOMER` table, both `customer_id` and `email` are candidate keys. We selected `customer_id` as the Primary Key and enforced `email` as an alternate key using a `UNIQUE` constraint."*

Q26. What is a Surrogate Key versus a Natural Key?

Spoken Answer: *"A natural key is an attribute that exists inherently in the real-world domain, like a customer's email or a movie title. A surrogate key is an artificially generated identifier, such as an `AUTO_INCREMENT INT` (`movie_id`, `booking_id`). We use surrogate keys for faster integer index joins and to isolate the database from*

real-world data changes."

Q27. What is a Junction Table (Associative Entity), and why is it necessary?

Spoken Answer: *"A junction table maps a Many-to-Many relationship between two tables into two One-to-Many relationships. Relational databases cannot directly implement M:N relationships without creating repeating groups or multi-valued columns.* `BOOKED_SEAT` *is our junction table connecting* `BOOKING` *and* `SEAT`*."*

Q28. How did you map the 1:1 relationship between CUSTOMER and ACCOUNT?

Spoken Answer: *"We placed* `customer_id` *as a foreign key inside the* `ACCOUNT` *table and applied a* `UNIQUE` *constraint (* `uq_account_customer`*). This guarantees that each customer can have at most one login account, enforcing strict 1:1 cardinality."*

Q29. How is the 1:1 relationship between BOOKING and PAYMENT enforced?

Spoken Answer: *"In the* `PAYMENT` *table,* `booking_id` *is defined with a* `UNIQUE` *constraint (* `uq_payment_booking`*) and references* `BOOKING(booking_id)`*. This ensures that each booking ticket can have exactly one financial settlement record."*

Q30. Explain the cardinality between THEATER and SCREEN.

Spoken Answer: *"It is a One-to-Many ($1:N$) relationship. One theater complex contains multiple screens ($1 \dots N$), but each screen belongs to exactly one physical theater complex ($1:1$). The foreign key* `theater_id` *resides in* `SCREEN`*."*

Q31. Explain the cardinality between SCREEN and SHOWTIME.

Spoken Answer: *"It is One-to-Many ($1:N$). An auditorium screen hosts many showtimes over time ($1 \dots N$), but each specific screening event occurs in exactly one auditorium ($1:1$). The foreign key* `screen_id` *resides in* `SHOWTIME`*."*

Q32. Explain the cardinality between MOVIE and SHOWTIME.

Spoken Answer: *"It is One-to-Many ($1:N$). One movie title can be scheduled across multiple showtimes in various screens and dates, but each scheduled showtime features exactly one movie. The foreign key* `movie_id` *resides in* `SHOWTIME`*."*

Q33. What does Crow's Foot notation represent in your ER diagram?

Spoken Answer: *"Crow's Foot notation visually denotes relationship cardinality. A single vertical line represents 'one', a circle represents 'zero' (optionality), and a three-pronged fork (crow's foot) represents 'many'. For example, the line from* `SCREEN` *to* `SEAT` *ends with a crow's foot, representing that one screen has many seats."*

Q34. What is Referential Integrity?

Spoken Answer: *"Referential integrity is a database rule ensuring that a foreign key value in one table must always match an existing primary key value in the referenced table, or be null. It prevents invalid pointers and orphan records."*

Q35. What is an Orphan Record, and how does your database prevent them?

Spoken Answer: *"An orphan record is a child row whose foreign key references a parent row that no longer exists (for example, a showtime referencing a deleted movie). CINEVERSE prevents orphans using* `ON DELETE RESTRICT` *on critical relationships like* `SHOWTIME -> MOVIE` *and* `BOOKING -> CUSTOMER`*."*

Section D: Normalization & Functional Dependencies

Q36. What is Normalization, and what is its primary objective?

Spoken Answer: *"Normalization is a systematic technique for organizing relational database tables to minimize data redundancy and eliminate update, insertion, and deletion anomalies while preserving data dependencies and lossless joins."*

Q37. What is First Normal Form (1NF)? Does CINEVERSE satisfy 1NF?

Spoken Answer: *"A relation is in 1NF if every attribute contains only atomic (indivisible) values, each column has a unique name, and there are no repeating groups. CINEVERSE satisfies 1NF because seats are never stored as comma-separated strings inside* `BOOKING`*; each seat is stored as an individual row in* `BOOKED_SEAT`*."*

Q38. What is a Functional Dependency? Give an example from your project.

Spoken Answer: *"A functional dependency $X \rightarrow Y$ means that the value of attribute X uniquely determines the value of attribute Y. In our* `CUSTOMER` *table,* `customer_id` *$\rightarrow$* `(customer_name, phone, email)`*. Knowing the* `customer_id` *uniquely identifies the customer's name, phone, and email."*

Q39. What is Second Normal Form (2NF)?

Spoken Answer: *"A relation is in 2NF if it is in 1NF and no non-prime attribute is partially dependent on any candidate key. That is, every non-key attribute must depend on the* entire *primary key, not a subset."*

Q40. Demonstrate how CINEVERSE satisfies 2NF using BOOKED_SEAT.

Spoken Answer: *"*`BOOKED_SEAT` *has a composite primary key:* `(booking_id, seat_id)`*. Its non-key attribute is* `ticket_price`*. The* `ticket_price` *depends on* both *the booking instance and the specific physical seat tier. It does not depend solely on* `booking_id` *or* `seat_id` *alone. Therefore, there are no partial dependencies, satisfying 2NF."*

Q41. Demonstrate how SNACK_ORDER_ITEM satisfies 2NF.

Spoken Answer: *"*`SNACK_ORDER_ITEM` *has a composite primary key:* `(snack_order_id, snack_id)`*. Non-key attributes are* `quantity` *and* `unit_price`*. Both attributes describe the line item and depend on both the specific order and the specific snack. Hence, it is fully in 2NF."*

Q42. What is Third Normal Form (3NF)?

Spoken Answer: *"A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key via another non-prime attribute ($X \rightarrow Y$ and $Y \rightarrow Z$ where Z is non-key)."*

Q43. Demonstrate how CINEVERSE satisfies 3NF using the SCREEN and THEATER tables.

Spoken Answer: *"In* `SCREEN`*,* `screen_id`*,* `screen_number`*,* `screen_name`*,* `screen_type`*,* `capacity`*, and* `theater_id`*. Notice that theater details—such as* `theater_name`*,* `city`*, and* `address` *—are* not *stored in* `SCREEN`*. If they were, we would have* `screen_id` *$\rightarrow$* `theater_id` *and* `theater_id` *$\rightarrow$* `city`*, a transitive dependency. By isolating theater attributes in* `THEATER`*, we eliminate transitive dependencies and satisfy 3NF."*

Q44. What is Boyce-Codd Normal Form (BCNF)?

Spoken Answer: *"BCNF is a stricter version of 3NF. A relation is in BCNF if, for every non-trivial functional dependency $X \rightarrow Y$, the determinant X is a superkey. All of our tables with single-column surrogate*

primary keys naturally satisfy BCNF."

Q45. What is an Insertion Anomaly? Give an example of how your design prevents it.

Spoken Answer: *"An insertion anomaly occurs when certain attributes cannot be inserted without the presence of other unrelated attributes. For example, if movies and showtimes were merged into one table, you could not insert an upcoming movie into the catalog until a showtime was scheduled. Our design separates* `MOVIE` *from* `SHOWTIME`*, allowing us to add movies with zero scheduled showtimes."*

Q46. What is a Deletion Anomaly?

Spoken Answer: *"A deletion anomaly occurs when deleting one piece of data unintentionally removes unrelated data. If theaters and screens were combined, deleting the last screen in a venue would accidentally delete the entire theater's address and record. In CINEVERSE, deleting a screen preserves the* `THEATER` *record."*

Q47. What is an Update Anomaly?

Spoken Answer: *"An update anomaly occurs when data redundancy forces the same value to be updated in multiple rows, risking data inconsistency. If theater addresses were duplicated across every showtime row, changing an address would require updating thousands of records. In 3NF, the address exists in exactly one row in* `THEATER`*."*

Q48. Is there any intentional Denormalization in CINEVERSE?

Spoken Answer: *"Yes, in* `BOOKED_SEAT.ticket_price`*. While the base price originates in* `SHOWTIME`*, recording* `ticket_price` *in* `BOOKED_SEAT` *represents an immutable financial snapshot at the time of purchase. If ticket base prices change tomorrow, past booking revenues remain historically accurate."*

Q49. What is a Lossless Join Decomposition?

Spoken Answer: *"A decomposition of relation R into R_1 and R_2 is lossless if joining R_1 and R_2 on their common attribute yields the exact original relation R without generating spurious tuples. In our schema, $R_1 \cap R_2$ always forms a superkey for at least one relation, guaranteeing lossless joins."*

Q50. What is Dependency Preservation?

Spoken Answer: *"Dependency preservation means that all functional dependencies of the original relation can be enforced simply by checking the constraints on the individual decomposed tables, without needing to perform a join. Our schema preserves all dependencies."*

Section E: SQL, MySQL Engine & Relational Theory

Q51. What are the five major categories of SQL commands?

Spoken Answer: *"1.* ***DDL (Data Definition Language):*** `CREATE`, `ALTER`, `DROP`, `RENAME`, `TRUNCATE`.
DML (Data Manipulation Language): `INSERT`, `UPDATE`, `DELETE`.

DQL (Data Query Language): `SELECT`.

DCL (Data Control Language): `GRANT`, `REVOKE`.

TCL (Transaction Control Language): `COMMIT`, `ROLLBACK`, `SAVEPOINT`."

Q52. What is the difference between TRUNCATE and DELETE?

Spoken Answer: *" `DELETE` is a DML command that removes rows one by one, logs individual row deletions in the transaction log, fires delete triggers, and can be rolled back. `TRUNCATE` is a DDL command that deallocates the table's data pages, resets auto-increment counters, is faster, and cannot be rolled back in some configurations."*

Q53. What is an INNER JOIN versus a LEFT OUTER JOIN?

Spoken Answer: *"An `INNER JOIN` returns only rows that have matching values in both tables. A `LEFT JOIN` returns all rows from the left table, and the matched rows from the right table; if no match exists, NULL values are returned for the right table's columns. In CINEVERSE, we use `LEFT JOIN BOOKING` when querying all customers so that customers with zero bookings are still included."*

Q54. What are Aggregate Functions in SQL? Name the five core functions.

Spoken Answer: *"Aggregate functions perform a calculation on a set of rows and return a single summary value. The five core functions are: `COUNT()`, `SUM()`, `AVG()`, `MIN()`, and `MAX()`. In `reportController.js`, we use `SUM(total_amount)` to compute gross theater revenue."*

Q55. What is the difference between WHERE and HAVING clauses?

Spoken Answer: *" `WHERE` filters individual rows* before *any grouping or aggregation takes place. `HAVING` filters aggregated groups* after *the `GROUP BY` clause has been evaluated. For example, `WHERE booking_status = 'CONFIRMED' GROUP BY movie_id HAVING COUNT( ) > 5`."*

Q56. What are the ACID properties in database transactions?

Spoken Answer: "- **Atomicity:** All statements in the transaction succeed, or the entire transaction is rolled back ('all or nothing').

Consistency: The database transitions from one valid state to another, satisfying all schema constraints.

Isolation: Concurrent transactions execute without interfering with one another.

Durability: Once committed, changes are permanently written to non-volatile storage and survive power failures."

Q57. How are ACID properties maintained during CINEVERSE ticket booking?

Spoken Answer: "In `createBooking`, we execute `await conn.beginTransaction()`. If seat validation, customer check, seat insertion, payment simulation, or snack deduction fails, `await conn.rollback()` is triggered. Changes are only made permanent when `await conn.commit()` executes."

Q58. What is a Clustered Index versus a Secondary (Non-Clustered) Index in MySQL InnoDB?

Spoken Answer: "In InnoDB, the primary key forms the **Clustered Index**, meaning table rows are physically stored on disk in the B+ Tree order of the primary key. A **Secondary Index** stores the index column and points to the clustered index primary key. When querying by secondary index, InnoDB looks up the secondary index first, then traverses the clustered index (index lookup)."

Q59. What secondary indexes exist in your CINEVERSE database?

Spoken Answer: "Our foreign keys and search columns have secondary B-Tree indexes:

`idx_showtime_date` on `SHOWTIME(show_date)`

`idx_showtime_movie` on `SHOWTIME(movie_id)`

`idx_showtime_screen` on `SHOWTIME(screen_id)`

`idx_booking_customer` on `BOOKING(customer_id)`

`idx_booking_showtime` on `BOOKING(show_id)`

`idx_booking_status` on `BOOKING(booking_status)`"

Q60. What is a Subquery? What is the difference between a Correlated and Non-Correlated subquery?

Spoken Answer: "A subquery is a `SELECT` query nested inside another query. A **non-correlated subquery** executes once independently, and its result is used by the outer query. A **correlated subquery** references columns from the outer query and must be evaluated repeatedly for each row processed by the outer query."

Q61. What does the COALESCE() function do?

Spoken Answer: "`COALESCE()` accepts a list of arguments and returns the first non-NULL value. In our revenue reporting queries, we write `COALESCE(SUM(b.total_amount), 0.00)` so that if a movie has zero confirmed bookings, the query returns `0.00` instead of `NULL`."

Q62. Does your live database have any Triggers, Views, or Stored Procedures?

Spoken Answer: "No, live schema introspection confirms `TRIGGERS: []`, `PROCEDURES: []`, and `VIEWS: []`. In CINEVERSE, business validations and multi-statement workflows are implemented in the Express backend controllers (`bookingController.js`, `showtimeController.js`) for better code versioning, unit testing, and horizontal scalability."

Q63. If asked to write a trigger, how would you write a trigger to update snack stock after an order?

Spoken Answer: "

```
CREATE TRIGGER trg_after_snack_item_insert
AFTER INSERT ON SNACK_ORDER_ITEM
FOR EACH ROW
BEGIN
    UPDATE SNACK
    SET stock_quantity = stock_quantity - NEW.quantity
    WHERE snack_id = NEW.snack_id;
END;
```

"

Q64. If asked to write a Stored Procedure, how would you write one to get a movie's revenue?

Spoken Answer: "

```
CREATE PROCEDURE sp_get_movie_revenue(IN p_movie_id INT, OUT p_revenue DECIMAL(10,2))
BEGIN
    SELECT COALESCE(SUM(b.total_amount), 0.00) INTO p_revenue
    FROM BOOKING b
    JOIN SHOWTIME s ON b.show_id = s.show_id
    WHERE s.movie_id = p_movie_id AND b.booking_status = 'CONFIRMED';
END;
```

"

Q65. What is the SQL CHECK constraint? Give two examples from CINEVERSE.

Spoken Answer: "A `CHECK` constraint specifies a boolean condition that every row in the table must satisfy. In CINEVERSE:
`chk_movie_rating`: `CHECK (rating >= 0.0 AND rating <= 10.0)` prevents invalid IMDb scores.
`chk_booking_status`: `CHECK (booking_status IN ('CONFIRMED', 'CANCELLED', 'PENDING'))` prevents invalid lifecycle states."

Section F: Code Walkthrough & Database Implementation

Q66. Where is the database connection configured in the backend?

Spoken Answer: "In `backend/src/config/db.js`. It exports `pool`, `query(sql, params)`, `getConnection()`, and `checkDbConnection()`. The pool uses `mysql2/promise` with configuration pulled from `process.env`."

Q67. How are SQL injection attacks prevented in your backend?

Spoken Answer: "We strictly execute parameterized queries using the `?` placeholder syntax in `pool.execute(sql, params)`. The `mysql2` driver prepares the statement on the database server using MySQL's binary protocol, separating the SQL syntax tree from the data inputs. Malicious strings like `' OR '1'='1` are treated purely as literal strings, completely neutralizing SQL injection."

Q68. What function handles showtime conflict prevention, and in which file?

Spoken Answer: "Function `adminCreateShowtime` in `backend/src/controllers/showtimeController.js` (lines 176–260). It queries for overlapping screenings on the same screen using interval overlap logic."

Q69. What mathematical condition detects an auditorium scheduling conflict?

Spoken Answer: "Two time intervals $[A_{start}, A_{end}]$ and $[B_{start}, B_{end}]$ overlap if and only if: $A_{start} < B_{end} \land A_{end} > B_{start}$. In SQL, we write: `WHERE s.screen_id = ? AND s.show_date = ? AND (s.start_time < ? AND s.end_time > ?)` where the parameters are the calculated end time and requested start time."

Q70. How does the backend calculate a screening's end time?

Spoken Answer: "In `showtimeController.js`, helper function `computeEndTime` takes `start_time`, `movie.duration` (in minutes), and adds a mandatory 15-minute turnaround buffer (`turnaroundMins = 15`) to allow auditorium cleaning between audiences."

Q71. How does the backend calculate dynamic seat pricing?

Spoken Answer: "In `bookingController.js` (lines 234–244), the server fetches `base_price` from `SHOWTIME`. It then iterates through the verified seats:
`Standard` seat: `base_price`
`Premium` seat: `Math.round(base_price * 1.2)` *(+20%)*
`VIP` / `Recliner`: `Math.round(base_price * 1.4)` (+40%).
Pricing is strictly calculated server-side; client-submitted totals are ignored."

Q72. Where are administrative dashboard metrics calculated?

Spoken Answer: "In `backend/src/controllers/adminController.js` under `getAdminDashboard`. It executes aggregate count queries on `MOVIE`, `SHOWTIME`, `BOOKING`, and `CUSTOMER`."

Q73. How does password hashing work for admin staff and customer accounts?

Spoken Answer: "We use `bcryptjs`. When creating accounts, plain passwords are salted with a work factor of 10 (`bcrypt.hash(password, 10)`). During login, `bcrypt.compare(password, staff.password_hash)` verifies the hash without ever storing or logging plain-text passwords."

Q74. How does the system handle booking cancellation?

Spoken Answer: "In `bookingController.js` under `cancelBooking`, it executes an atomic transaction with a row lock: `SELECT booking_id FROM BOOKING WHERE booking_id = ? FOR UPDATE`. It updates `BOOKING.booking_status = 'CANCELLED'` and `PAYMENT.payment_status = 'REFUNDED'`. This immediately frees the seats for other customers while maintaining the transaction log."

Q75. How are database errors handled across the API?

Spoken Answer: "All async controller functions wrap their logic in `try ... catch` blocks and forward caught errors to `next(err)`. In `server.js`, an Express centralized error-handling middleware catches `err`, checks MySQL error codes (such as `ER_DUP_ENTRY` for duplicate emails or `ER_ROW_IS_REFERENCED_2` for foreign key violations), and returns clean, structured JSON error messages."

Section G: Movie, Theatre & Showtime Management

Q76. How does the system prevent an auditorium from being double-booked for two movies at the same time?

Spoken Answer: "When an admin attempts to schedule a screening, `adminCreateShowtime` checks the database for any existing showtime on that screen and date where $(s_{start} < end_{calc} \land s_{end} > start_{form})$. If a conflict is found, the server rejects the request with HTTP 409 Conflict and names the conflicting movie and time slot."

Q77. Can a movie exist in the database without any showtimes?

Spoken Answer: "Yes. In our relational schema, `SHOWTIME` holds the foreign key `movie_id` referencing `MOVIE`. `MOVIE` does not hold a foreign key to `SHOWTIME`. Thus, movies can be inserted into the catalog months ahead of release."

Q78. Can a showtime exist without a movie or screen?

Spoken Answer: "No. In `SHOWTIME`, both `movie_id` and `screen_id` are defined as `INT NOT NULL` foreign keys. The MySQL engine will reject any showtime insertion where either identifier is null or references a non-existent row."

Q79. What happens if an admin tries to delete a movie that already has scheduled showtimes?

Spoken Answer: "The operation is rejected. First, the backend controller checks `SELECT show_id FROM SHOWTIME WHERE movie_id = ?` and returns HTTP 409. Second, at the database level, `fk_showtime_movie` enforces `ON DELETE RESTRICT`, throwing MySQL Error 1451."

Q80. How is auditorium screen seating configured?

Spoken Answer: "Screens have capacity ranging from 112 to 160 seats. Physical rows are assigned letters (A through K), and numbers (1 to 16). Rows A–D are `Standard`, E–H are `Premium`, and I–K are `VIP` / `Recliner`. The unique constraint `uq_screen_seat (screen_id, seat_number)` prevents duplicate seat labels in the same auditorium."

Q81. How does the customer website discover which movies are playing in a specific city?

Spoken Answer: "The customer frontend queries `GET /api/theaters`. When the user selects a city (e.g., 'Bengaluru'), the UI filters showtimes by joining `THEATER` on `city = 'Bengaluru'`, displaying only screenings occurring in multiplexes within that metropolitan area."

Q82. What is the difference between screen capacity and seat count?

Spoken Answer: "`capacity` is an integer column on `SCREEN` indicating total permitted audience. `SEAT` contains the individual physical rows for that screen. In our database, the count of rows in `SEAT WHERE screen_id = X` exactly equals `SCREEN.capacity`."

Q83. What is movie release status, and how is it derived?

Spoken Answer: "A movie is 'Now Showing' if it has at least one active, scheduled showtime in the database. It is 'Coming Soon' if its release date is in the future and it has zero current showtimes. Release status is dynamically derived from relational showtime counts."

Q84. Why does the `MOVIE` table store `duration` as an integer?

Spoken Answer: "It stores duration in total minutes (e.g., 180 for Oppenheimer). Storing duration as an integer allows simple arithmetic addition with start times and turnaround buffers to compute precise screening end times."

Q85. How do you prevent deleting a theater that has active staff assigned?

Spoken Answer: "The foreign key constraint `fk_admin_theater` on `ADMIN_STAFF(theater_id)` is configured with `ON DELETE RESTRICT`. MySQL prevents deleting the theater record until all assigned staff members are reassigned or removed."

Section H: Seat Selection, Booking Logic & Concurrency

Q86. How are physical seats mapped to a booking in the database?

Spoken Answer: "Via the associative junction table `BOOKED_SEAT`. For every seat selected in a booking, a row is inserted with `(booking_id, seat_id, ticket_price)`. The composite primary key `(booking_id, seat_id)` prevents the same seat from being inserted twice within the same booking."

Q87. How does the system calculate how many seats are available for a showtime?

Spoken Answer: "In `getShowtimeSeats`, the query counts all physical seats for the screen: `SELECT COUNT() FROM SEAT WHERE screen_id = ?`. *It then queries:* `SELECT bs.seat_id FROM BOOKED_SEAT bs JOIN BOOKING b ON bs.booking_id = b.booking_id WHERE b.show_id = ? AND b.booking_status != 'CANCELLED'`. *Available seats = Total Seats $-$ Booked Seats."*

Q88. How does the backend prevent a user from booking a seat that belongs to a different screen?

Spoken Answer: *"In Step 4 of `createBooking`, the server queries the `SEAT` table for all requested `seat_ids` and verifies that every returned seat's `screen_id` matches the showtime's assigned `screen_id`. If a mismatch is*

detected, the transaction aborts with HTTP 400."

Q89. How does the system prevent two users from booking the same seat for the same showtime?

Spoken Answer: *"In Step 5 of* `createBooking`*, within an active transaction, the server queries* `BOOKED_SEAT` *joined with* `BOOKING` *for the target* `show_id` *and selected* `seat_ids` *where* `booking_status != 'CANCELLED'`*. If any seats are returned, the transaction rolls back with HTTP 409 Conflict ('Seats already booked')."*

Q90. What is a race condition in seat reservation? Does your current implementation completely prevent it?

Spoken Answer (Honest & Defensible): *"A race condition occurs when two concurrent requests read seat availability simultaneously before either writes. In our current implementation, the seat conflict query uses standard* `SELECT` *without* `FOR UPDATE`*. Under high concurrent load, two requests could theoretically read the seats as available at the exact same millisecond. To make it 100% race-condition proof, we should add* `SELECT ... FOR UPDATE` *on the seat rows or enforce a unique constraint on* `(show_id, seat_id)`*."*

Q91. What is the purpose of the `FOR UPDATE` *clause in SQL?*

Spoken Answer: *"*`SELECT ... FOR UPDATE` *acquires an exclusive row-level lock on the read rows within an InnoDB transaction. Any other concurrent transaction attempting to read or update those locked rows is blocked until the first transaction commits or rolls back. In CINEVERSE, we use* `FOR UPDATE` *on* `SNACK` *inventory and* `BOOKING` *cancellation."*

Q92. What are the four ANSI SQL Transaction Isolation Levels?

Spoken Answer: *"1.* ***Read Uncommitted:*** *Permitted dirty reads.*
Read Committed: *Prevents dirty reads; non-repeatable reads possible.*

Repeatable Read: *(MySQL InnoDB default) Prevents dirty and non-repeatable reads; phantom reads mitigated by Next-Key Locking.*

Serializable: *Highest isolation; fully serializes access using range locks."*

Q93. What is a Dirty Read, Non-Repeatable Read, and Phantom Read?

Spoken Answer: *"-* ***Dirty Read:*** *A transaction reads uncommitted changes made by another transaction that might later be rolled back.*
Non-Repeatable Read: *A transaction reads the same row twice and observes different data because another transaction committed an update in between.*

Phantom Read: *A transaction re-executes a query returning a range of rows and finds new 'phantom' rows inserted and committed by another transaction."*

Q94. Why don't you delete rows from `BOOKED_SEAT` *when a booking is cancelled?*

Spoken Answer: *"Because* `BOOKING.booking_status` *is updated to* `'CANCELLED'`*. Our seat availability queries filter* `WHERE b.booking_status != 'CANCELLED'`*. By leaving the rows in* `BOOKED_SEAT`*, the seats are immediately released for new bookings while preserving complete historical audit trails of who originally reserved the seats."*

Q95. How does the frontend seat map visually distinguish seat categories?

Spoken Answer: *"The customer frontend seat picker fetches seat objects with* `seat_type`*. The UI assigns distinct visual color schemes: Silver for Standard, Gold for Premium, and Purple/Rose for VIP Recliners, displaying*

corresponding price badges."

Section I: Payments & Concession Orders

Q96. How is concession (snack) ordering integrated with ticket booking?

Spoken Answer: *"Snack orders are tied directly to the ticket reservation via `SNACK_ORDER(booking_id)`. The grand total of the booking combines the ticket total and snack total. Concession line items are stored in `SNACK_ORDER_ITEM`."*

Q97. How does CINEVERSE prevent selling snacks that are out of stock?

Spoken Answer: *"In Step 7 of `createBooking`, for every snack ordered, the server executes: `SELECT stock_quantity FROM SNACK WHERE snack_id = ? FOR UPDATE`. If `stock_quantity < requested_quantity`, the transaction rolls back immediately with HTTP 400 ('Insufficient stock'). If sufficient, it decrements inventory: `UPDATE SNACK SET stock_quantity = stock_quantity - ? WHERE snack_id = ?`."*

Q98. Is the payment gateway live or simulated?

Spoken Answer: *"It is an internal transactional simulation. The `PAYMENT` table records the transaction amount, method (`UPI`, `Credit Card`, `Debit Card`, `Net Banking`, `Cash`), timestamp, and status (`COMPLETED`). The relational integrity and financial reconciliation are fully modeled, but without external bank API webhooks."*

Q99. What happens to the payment record if a booking is cancelled?

Spoken Answer: *"In `cancelBooking`, the payment record's status is updated: `UPDATE PAYMENT SET payment_status = 'REFUNDED' WHERE booking_id = ?`. This preserves financial accountability for auditing purposes."*

Q100. How does the schema guarantee that one booking cannot have two conflicting payments?

Spoken Answer: *"The `PAYMENT` table defines a unique constraint `uq_payment_booking UNIQUE (booking_id)`. Any attempt to insert a duplicate payment receipt for the same booking ID is rejected by the database engine."*

Q101. What payment methods are supported by database constraints?

Spoken Answer: *"The check constraint `chk_payment_method` restricts values to: `'UPI'`, `'Credit Card'`, `'Debit Card'`, `'Net Banking'`, or `'Cash'`. Inserting any other string throws a check constraint violation."*

Q102. How do you calculate total concession revenue versus ticket revenue?

Spoken Answer: *"In `reportController.js`:*
Ticket Revenue: `SELECT SUM(total_amount) FROM BOOKING WHERE booking_status = 'CONFIRMED'`.
Concession Revenue: `SELECT SUM(total_price) FROM SNACK_ORDER WHERE order_status = 'COMPLETED'`."

Section K: Testing, Debugging & Edge Cases

Q103. How was the CINEVERSE application tested?

Spoken Answer: *"We tested the application through 37 automated integration tests in* `scratch/comprehensive_test.js` *covering database release statuses, scheduling collision detection, admin trailer CRUD, and multi-seat transaction rollback. We also verified frontend production builds with zero compiler errors."*

Q104. What happens if the MySQL database server crashes or loses connection?

Spoken Answer: *"In* `backend/src/config/db.js`, `checkDbConnection` *detects connection failures, captures the error code (* `ECONNREFUSED`*), and logs a structured diagnostic message. The API endpoints return HTTP 500 with a clean error message rather than crashing the Node.js process."*

Q105. What happens if a user submits a negative ticket price or negative snack quantity?

Spoken Answer: *"The operation is rejected at two levels: First, controller input validation checks* `if (price <= 0) return res.status(400)`*. Second, MySQL schema constraints* `chk_showtime_base_price (base_price >= 0.00)` *and* `chk_snack_item_qty (quantity > 0)` *enforce non-negative values at the engine level."*

Q106. What happens if a customer attempts to book zero seats?

Spoken Answer: *"In* `bookingController.js` *line 142, the server validates:* `if (!seat_ids || !Array.isArray(seat_ids) || seat_ids.length === 0)`*. It immediately rejects the request with HTTP 400 Bad Request before opening a database transaction."*

Q107. How did you debug SQL syntax and execution errors during development?

Spoken Answer: *"We monitored Node.js server console logs and MySQL query logs. We isolated queries and tested them directly in the MySQL command line client using* `EXPLAIN` *to inspect query execution plans, verify index usage, and measure execution times."*

Q108. What is the output of `EXPLAIN SELECT ...` *in MySQL?*

Spoken Answer: *"* `EXPLAIN` *displays the query execution plan chosen by MySQL's query optimizer. Key columns include:*

`type` *: Join type (* `ALL` *full table scan,* `ref` *index lookup,* `eq_ref` *unique index lookup).*

`possible_keys` *: Indexes MySQL could use.*

`key` *: The actual index chosen.*

`rows` *: Estimated number of rows MySQL must examine."*

Q109. What happens if an admin schedules a showtime with end time before start time?

Spoken Answer: *"The scheduling collision engine computes* `end_time = computeEndTime(start_time, duration, 15)`*. Since duration is constrained by* `chk_movie_duration (duration > 0)`*, the calculated end time is strictly guaranteed to be after the start time."*

Q110. How did you verify referential integrity under cascading deletes?

Spoken Answer: *"In a test environment, deleting a test* `SCREEN` *automatically cascade-deleted all associated* `SEAT` *rows (* `ON DELETE CASCADE`*), while attempting to delete a* `MOVIE` *with active showtimes was blocked (* `ON DELETE RESTRICT`*)."*

Section L: Security, Reliability & Performance

Q111. How does CINEVERSE prevent Cross-Site Scripting (XSS)?

Spoken Answer: *"React automatically escapes all variables rendered inside JSX expressions* `{movie.title}`*, treating input as plain strings rather than executable HTML. Additionally, backend inputs are validated and sanitized."*

Q112. How does the system handle Admin Authentication?

Spoken Answer: *"In* `adminController.js`*, when staff log in with email and password, the server verifies the bcrypt salted password hash. Upon success, it issues a signed JSON Web Token (JWT) containing* `staff_id`*,* `email`*, and* `role`*, valid for 8 hours."*

Q113. How does the backend authorize protected administrative routes?

Spoken Answer: *"Using JWT verification middleware in* `backend/src/middleware/auth.js`*. Protected routes inspect the* `Authorization: Bearer <token>` *header. If the token is missing, expired, or tampered with, the request is rejected with HTTP 401 Unauthorized."*

Q114. How would you optimize query performance if your database grew to 10 million bookings?

Spoken Answer: *"1.* ***Partitioning:*** *Partition the* `BOOKING` *and* `BOOKED_SEAT` *tables by range on* `booking_date` *(e.g., yearly/monthly partitions).*
Composite Indexing: *Add composite indexes on* `(show_id, booking_status)`*.*
Archiving: *Move completed bookings older than 2 years into a cold historical archive table.*
Caching: *Cache movie catalog and theater venue data in Redis."*

Q115. Why is connection pooling crucial for backend reliability?

Spoken Answer: *"Creating a new database connection for every HTTP request incurs heavy TCP handshake and authentication overhead, exhausting database ports under traffic spikes. A connection pool reuses active connections, capping database load and preventing out-of-memory crashes."*

Q116. What sensitive customer data is stored, and how is it protected?

Spoken Answer: *"We store customer name, phone, email, and password hash. Password hashes use salted bcrypt (* `$2a$10$...`*), rendering them resistant to rainbow table and brute-force attacks. No raw credit card or CVV numbers are ever stored."*

Q117. What is Database Connection Starvation, and how do you prevent it?

Spoken Answer: *"Connection starvation occurs when long-running queries or unreleased connections exhaust the pool, causing incoming requests to hang indefinitely. We prevent this by strictly using* `try ... finally { conn.release(); }` *to guarantee that connections return to the pool immediately upon request completion."*

Q118. What is Deadlock in MySQL, and how does InnoDB handle it?

Spoken Answer: *"A deadlock occurs when two transactions hold locks that the other needs, creating a circular wait where neither can proceed. InnoDB automatically detects deadlocks using a wait-for graph, immediately aborts the transaction with fewer row locks (the lower-cost transaction), and rolls it back, allowing the other to complete."*

Section M: Critical & Difficult Examiner Questions

Q119. Why did you choose MySQL instead of PostgreSQL?

Spoken Answer: *"MySQL 8.0 with InnoDB provides enterprise ACID compliance, native B-Tree indexing, robust foreign key constraint enforcement, and straightforward replication. While PostgreSQL offers advanced object-relational features, MySQL is the industry standard for high-throughput commercial web backends and perfectly fits our relational ticketing model."*

Q120. If an examiner claims: "Your seat booking logic is flawed because two users can book the same seat simultaneously," how do you respond?

Spoken Answer (Defensible & Masterful): *"You are highlighting an essential concurrency consideration. In our current implementation, we execute an application-level conflict check inside a multi-statement transaction, which handles typical sequential traffic. However, because our conflict query does not use `SELECT ... FOR UPDATE`, under extreme millisecond concurrency, two transactions could experience a race condition under MySQL's default `REPEATABLE READ` isolation. To solve this completely, we can add `FOR UPDATE` to lock the seat rows during validation or add an engine-level unique key constraint on `(show_id, seat_id)`."*

Q121. What happens if payment succeeds but booking creation fails?

Spoken Answer: *"In our architecture, payment recording and booking creation occur within the* exact same* ACID transaction. If booking creation fails, the transaction rolls back before payment is committed. In a commercial deployment with an external payment gateway, we would implement a Two-Phase Commit or Webhook Reconciliation worker: if a payment succeeds but the booking fails, the system automatically triggers an immediate gateway refund."

Q122. Why did you use surrogate integer keys (`movie_id`, `show_id`) instead of natural keys (`title`, `start_time`)?

Spoken Answer: "Natural keys like movie titles or timestamps are wide strings that consume significant index page space, degrade B-Tree performance, and can change in the real world (e.g., a movie title correction). Surrogate integer keys are compact (4 bytes), immutable, highly performant for B-Tree index traversal, and simplify multi-table joins."

Q123. Can your database schema support ticket price discounts or promo coupons?

Spoken Answer: "Yes. We can introduce a `DISCOUNT_COUPON` table with `(coupon_id, coupon_code, discount_pct, expiry_date)` and add an optional nullable foreign key `coupon_id` to `BOOKING`. The total amount in `BOOKING` would reflect the net price after coupon deduction."

Q124. How would your schema handle seat reservation for users with disabilities (wheelchair access)?

Spoken Answer: "The `SEAT` table already includes a `seat_type` column constrained by `chk_seat_type`. We can simply extend the allowed set in `chk_seat_type` to include `'Accessible'` or `'Wheelchair'` and flag these seats in the frontend with specialized icons and priority booking rules."

Q125. Why did you write raw SQL queries in your backend controllers instead of using an ORM like Prisma or Sequelize?

Spoken Answer: "Writing raw SQL queries via `mysql2/promise` was a deliberate academic choice. ORMs abstract away relational mechanics, often generating inefficient, bloated joins. Writing raw SQL allowed us to

explicitly design and demonstrate multi-table joins, aggregations, parameterized queries, and ACID transaction boundaries as mandated by the RDBMS laboratory syllabus."

Q126. What would you change in the database design if you had one more week?

Spoken Answer: "Three specific enhancements:
Add an engine-level unique constraint on `(show_id, seat_id)` or an ephemeral 5-minute reservation lock table to make seat selection 100% race-condition proof.

Implement database Views for complex reporting queries like `vw_movie_revenue_summary`.

Add a loyalty point redemption transaction table to complete the customer rewards loop."

Q127. Which part of your project demonstrates the strongest RDBMS concepts?

Spoken Answer: "The `createBooking` transaction and the junction table design for `BOOKED_SEAT`. It integrates 1NF-to-3NF normalization, composite primary keys, composite unique keys, foreign key cascading and restriction, server-side dynamic price calculations, and multi-statement ACID transaction rollback across three related tables."

Q128. Why does `BOOKING` have a `show_id` foreign key if `BOOKED_SEAT` already links to `SEAT`?

Spoken Answer: "Because `SEAT` represents a physical chair in an auditorium, not a screening event. A physical seat exists across hundreds of showtimes. Without `show_id` on `BOOKING`, it would be impossible to know which specific movie screening the customer purchased tickets for."

Q129. How does CINEVERSE protect against data loss in the event of a sudden power outage?

Spoken Answer: "Through MySQL InnoDB's write-ahead logging (WAL) mechanism known as the **Redo Log** (`ib_logfile`). Before changes are flushed to the main data tablespace, transactions are written sequentially to the redo log. Upon restarting after a crash, InnoDB reads the redo log and automatically rolls forward committed transactions while rolling back uncommitted ones."

Q130. What is the difference between Database Authentication and Application Authorization?

Spoken Answer: "Database Authentication verifies the Node.js backend's credentials (`root` user and password) when establishing a connection pool to MySQL. Application Authorization verifies the human user's identity and privileges (e.g., verifying a JWT token to determine if an admin staff member is authorized to schedule showtimes)."

CINEVERSE — Practical SQL Viva Voce Question Bank

Hands-On SQL Query Tasks & Code Breakdown Course: Relational Database Management Systems Laboratory (CSE Department) **Institution:** Manipal University Jaipur (MUJ) **Database:** MySQL 8.0 (`MovieTheaterDB`)

Guide for Practical SQL Viva

When an examiner asks you to write or explain an SQL query on paper or in MySQL Workbench:
Name the Target Tables First: Explain which tables contain the necessary attributes and how they join.

State Join Conditions Explicitly: Emphasize primary key to foreign key matches (`ON t1.id = t2.t1_id`).

Clarify Aggregate Filters: Distinguish between `WHERE` (row-level filter before grouping) and `HAVING` (group-level filter after aggregation).

Explain Output: Describe what each row of the result set represents in the cinema domain.

Task 1: List All Movies with Ratings and Runtimes

Examiner Prompt: "Write a query to retrieve all movies, showing their title, genre, runtime in minutes, and rating, ordered from highest rated to lowest rated." **SQL Query:**

```
SELECT
    movie_id,
    title,
    genre,
    duration AS duration_mins,
    rating
FROM MOVIE
ORDER BY rating DESC, title ASC;
```

Line-by-Line Explanation:

`SELECT movie_id, title, ...` *: Selects the relevant movie identification and descriptive attributes.*

`FROM MOVIE` *: Queries the base strong entity table storing the cinematic catalog.*

`ORDER BY rating DESC, title ASC` *: Sorts the catalog primarily by audience rating descending (highest rated first), using alphabetical title ascending as a tiebreaker.*

Expected Result: Returns 13 rows displaying titles like *The Dark Knight* (9.0), *Inception* (8.8), and *Interstellar* (8.7). ***Relational Concept:*** *Basic DQL projection, attribute aliasing, and multi-column sorting.*

Task 2: Count Screens and Total Auditorium Capacity Per Multiplex Theater

Examiner Prompt: *"Write a query to list each theater's name, city, total number of screens, and total seating capacity across all its screens."* ***SQL Query:***

```
SELECT
    t.theater_id,
    t.theater_name,
    t.city,
    COUNT(s.screen_id) AS total_screens,
    COALESCE(SUM(s.capacity), 0) AS total_theater_capacity
FROM THEATER t
LEFT JOIN SCREEN s ON t.theater_id = s.theater_id
GROUP BY t.theater_id, t.theater_name, t.city
```

```
  ORDER BY total_theater_capacity DESC;
```

Line-by-Line Explanation:

`SELECT t.theater_id, ...` : Selects the theater identifier, name, and city.

`COUNT(s.screen_id)` : Aggregates the total number of physical screens belonging to that theater.

`COALESCE(SUM(s.capacity), 0)` : Sums the capacity column across all associated screens, defaulting to 0 if a theater has no screens.

`FROM THEATER t LEFT JOIN SCREEN s ON t.theater_id = s.theater_id` : Performs a Left Outer Join so theaters with zero screens are not excluded.

`GROUP BY t.theater_id, t.theater_name, t.city` : Groups by all non-aggregated select columns to comply with SQL standard grouping rules.

Expected Result: *Returns 8 rows (one per multiplex). Each theater in Bengaluru, Mumbai, Delhi, Hyderabad, and Chennai shows 4 screens with total capacities around 500–600 seats.* **Relational Concept:** One-to-Many join, aggregate summation, `GROUP BY` , and `COALESCE` null-handling.

Task 3: Find All Available Seats for a Specific Showtime

Examiner Prompt: "Given a showtime ID (e.g., `show_id = 1`), write a query to list all physical seats that are currently available to be booked." **SQL Query:**

```
  SELECT
      st.seat_id,
      st.seat_number,
      st.seat_type
  FROM SHOWTIME sh
  JOIN SEAT st ON sh.screen_id = st.screen_id
  WHERE sh.show_id = 1
    AND st.seat_id NOT IN (
        SELECT bs.seat_id
        FROM BOOKED_SEAT bs
        JOIN BOOKING b ON bs.booking_id = b.booking_id
        WHERE b.show_id = 1
          AND b.booking_status != 'CANCELLED'
    )
  ORDER BY
      SUBSTRING(st.seat_number, 1, 1) ASC,
      CAST(SUBSTRING(st.seat_number, 2) AS UNSIGNED) ASC;
```

Line-by-Line Explanation:

`SELECT st.seat_id, ...` : Projects the seat identifier, row/number label, and tier.

`FROM SHOWTIME sh JOIN SEAT st ON sh.screen_id = st.screen_id` : Joins the showtime to its scheduled auditorium screen to retrieve all physical seats belonging to that auditorium.

`WHERE sh.show_id = 1` : Focuses on the target screening.

`AND st.seat_id NOT IN (...)` *: Subquery filtering out seats that already have an active, non-cancelled reservation for that specific showtime.*

`ORDER BY SUBSTRING(...)` *: Natural sort ordering letters (A to K) and numbers (1 to 16).*

Expected Result: Returns all unreserved seats in that auditorium. If the screen has 142 seats and 4 are booked, exactly 138 rows are returned. ***Relational Concept:*** *Set difference using subquery with* `NOT IN`*, correlated screening logic, and string sorting.*

Task 4: Total Revenue Generated by Each Movie

Examiner Prompt: *"Write a query to calculate total ticket revenue generated by each movie from confirmed bookings, ordered by revenue descending."* ***SQL Query:***

```
SELECT
    m.movie_id,
    m.title,
    m.genre,
    COUNT(DISTINCT b.booking_id) AS confirmed_bookings_count,
    COUNT(bs.seat_id) AS total_tickets_sold,
    COALESCE(SUM(b.total_amount), 0.00) AS gross_ticket_revenue
FROM MOVIE m
JOIN SHOWTIME s ON m.movie_id = s.movie_id
LEFT JOIN BOOKING b ON s.show_id = b.show_id AND b.booking_status = 'CONFIRMED'
LEFT JOIN BOOKED_SEAT bs ON b.booking_id = bs.booking_id
GROUP BY m.movie_id, m.title, m.genre
ORDER BY gross_ticket_revenue DESC;
```

Line-by-Line Explanation:

`SELECT m.movie_id, ...` : Projects movie details.

`COUNT(DISTINCT b.booking_id)` : Counts unique reservation orders to avoid counting duplicate bookings across multiple seats.

`COUNT(bs.seat_id)` : Counts individual tickets sold.

`COALESCE(SUM(b.total_amount), 0.00)` : Calculates total revenue, ensuring movies with zero bookings display 0.00 rather than NULL.

`LEFT JOIN BOOKING b ... AND b.booking_status = 'CONFIRMED'` : Only tallies valid confirmed revenue, excluding cancelled orders.

`GROUP BY m.movie_id, m.title, m.genre` : Standard grouping by primary key and attributes.

Expected Result: *Returns a list of movies ranked by gross ticket sales, led by popular blockbusters.* **Relational Concept:** Multi-table join across 4 tables, distinct counting, conditional join filtering, and aggregate summation.

Task 5: Customer Booking History and Invoice Summary

Examiner Prompt: "Write a query to fetch complete booking history for a specific customer (e.g., `customer_id = 1`), including movie title, show date, theater, screen number, payment method, and comma-separated seat numbers." **SQL Query:**

```
  SELECT
      b.booking_id,
      b.booking_date,
      m.title AS movie_title,
      t.theater_name,
      t.city,
      sc.screen_number,
      s.show_date,
      s.start_time,
      b.total_amount,
      b.booking_status,
      p.payment_method,
      p.payment_status,
      GROUP_CONCAT(st.seat_number ORDER BY st.seat_number SEPARATOR ', ') AS
reserved_seats
  FROM BOOKING b
  JOIN SHOWTIME s ON b.show_id = s.show_id
  JOIN MOVIE m ON s.movie_id = m.movie_id
  JOIN SCREEN sc ON s.screen_id = sc.screen_id
  JOIN THEATER t ON sc.theater_id = t.theater_id
  LEFT JOIN PAYMENT p ON b.booking_id = p.booking_id
  LEFT JOIN BOOKED_SEAT bs ON b.booking_id = bs.booking_id
  LEFT JOIN SEAT st ON bs.seat_id = st.seat_id
  WHERE b.customer_id = 1
  GROUP BY b.booking_id, b.booking_date, m.title, t.theater_name, t.city,
sc.screen_number, s.show_date, s.start_time, b.total_amount, b.booking_status,
p.payment_method, p.payment_status
  ORDER BY b.booking_date DESC;
```

Line-by-Line Explanation:

Connects `BOOKING` across 7 tables to reconstitute the full commercial invoice.

`GROUP_CONCAT(st.seat_number ...)` : Aggregates multiple individual seat rows (e.g., A1, A2) into a single readable string per booking invoice.

`WHERE b.customer_id = 1` : Restricts results to the targeted patron.

Expected Result: Returns all booking orders made by customer 1 with complete theater, timing, and seat assignment details. ***Relational Concept:*** *Complex multi-table equijoin, string aggregation with `GROUP_CONCAT`, and relational projection.*

Task 6: Detect Scheduling Conflicts on the Same Auditorium Screen

Examiner Prompt: *"Write a query to verify whether scheduling a screening on Screen ID 3 on date '2026-10-30'*

from '18:00:00' to '21:15:00' conflicts with any existing screening." ***SQL Query:***

```
SELECT
    s.show_id,
    m.title AS conflicting_movie,
    s.start_time,
    s.end_time
FROM SHOWTIME s
JOIN MOVIE m ON s.movie_id = m.movie_id
WHERE s.screen_id = 3
  AND s.show_date = '2026-10-30'
  AND (s.start_time < '21:15:00' AND s.end_time > '18:00:00');
```

Line-by-Line Explanation:

`WHERE s.screen_id = 3 AND s.show_date = '2026-10-30'` : Isolates screenings scheduled in the exact same physical auditorium on that calendar day.

`AND (s.start_time < '21:15:00' AND s.end_time > '18:00:00')` : The mathematical interval overlap condition. If any existing showtime starts before the candidate ends, AND ends after the candidate starts, an overlap exists.

Expected Result: *Returns empty result set if the time slot is clear, or returns the conflicting movie title and hours if occupied.* **Relational Concept:** Interval comparison logic, range overlap detection, and domain integrity enforcement.

Task 7: Concession Sales Revenue Breakdown by Snack Item

Examiner Prompt: "Write a query to calculate total quantity sold and total revenue earned for each snack item, showing top-selling items first." **SQL Query:**

```
SELECT
    sn.snack_id,
    sn.snack_name,
    sn.price AS current_unit_price,
    sn.stock_quantity AS remaining_stock,
    COALESCE(SUM(soi.quantity), 0) AS total_units_sold,
    COALESCE(SUM(soi.quantity  soi.unit_price), 0.00) AS total_concession_revenue
FROM SNACK sn
LEFT JOIN SNACK_ORDER_ITEM soi ON sn.snack_id = soi.snack_id
LEFT JOIN SNACK_ORDER so ON soi.snack_order_id = so.snack_order_id AND
so.order_status = 'COMPLETED'
GROUP BY sn.snack_id, sn.snack_name, sn.price, sn.stock_quantity
ORDER BY total_concession_revenue DESC;
```

Line-by-Line Explanation:

`FROM SNACK sn LEFT JOIN SNACK_ORDER_ITEM soi ...`: Left joins concessions so items with zero sales are still listed.

`COALESCE(SUM(soi.quantity soi.unit_price), 0.00)`: *Multiplies recorded line item quantities by historical purchase prices to calculate accurate historical revenue.*

`ORDER BY total_concession_revenue DESC`: *Sorts top revenue-earning snacks first.*

Expected Result: Returns 10 rows showing items like *Caramel Popcorn (L)* and *Cheese Nachos* leading in units and revenue. ***Relational Concept:*** *Many-to-Many junction aggregation, calculated column summation, and inventory auditing.*

Task 8: Find Customers with Zero Bookings (Inactive Patrons)

Examiner Prompt: *"Write a query to identify all registered customers who have never made a single ticket booking."* ***SQL Query (Method 1: LEFT JOIN with NULL check):***

```
SELECT
    c.customer_id,
    c.customer_name,
    c.email,
    c.phone
FROM CUSTOMER c
LEFT JOIN BOOKING b ON c.customer_id = b.customer_id
WHERE b.booking_id IS NULL;
```

Alternative Query (Method 2: NOT EXISTS Correlated Subquery):

```
SELECT
    c.customer_id,
    c.customer_name,
    c.email,
    c.phone
FROM CUSTOMER c
WHERE NOT EXISTS (
    SELECT 1
    FROM BOOKING b
    WHERE b.customer_id = c.customer_id
);
```

Line-by-Line Explanation:

Method 1 joins customers to bookings; if a customer has no bookings, the right table columns are `NULL`. `WHERE b.booking_id IS NULL` *filters strictly for those non-matching rows.*

Method 2 uses `NOT EXISTS`*, which returns true if the subquery returns zero rows for that customer.*

Expected Result: Returns customers in the database who registered an account or profile but have not yet

purchased tickets. ***Relational Concept:*** *Outer joins for non-existence matching versus correlated* `NOT EXISTS` *subqueries.*

Task 9: Overall Financial Reconciliation (Tickets, Snacks, Refunds)

Examiner Prompt: *"Write an executive financial summary query showing total confirmed ticket revenue, total concession revenue, total refunded amount, and net gross revenue."* ***SQL Query:***

```
  SELECT
      ticket_summary.confirmed_tickets,
      concession_summary.total_concessions,
      ticket_summary.refunded_amount,
      (ticket_summary.confirmed_tickets + concession_summary.total_concessions) AS
net_gross_revenue
  FROM
      (
          SELECT
              COALESCE(SUM(CASE WHEN booking_status = 'CONFIRMED' THEN total_amount
ELSE 0 END), 0.00) AS confirmed_tickets,
              COALESCE(SUM(CASE WHEN booking_status = 'CANCELLED' THEN total_amount
ELSE 0 END), 0.00) AS refunded_amount
          FROM BOOKING
      ) AS ticket_summary
  CROSS JOIN
      (
          SELECT
              COALESCE(SUM(total_price), 0.00) AS total_concessions
          FROM SNACK_ORDER
          WHERE order_status = 'COMPLETED'
      ) AS concession_summary;
```

Line-by-Line Explanation:
Uses two derived subqueries (`ticket_summary` and `concession_summary`).

Conditional `SUM(CASE ...)` separates confirmed ticket earnings from refunded amounts.

`CROSS JOIN` combines the two single-row scalar summaries to calculate overall net revenue.

Expected Result: *Returns exactly 1 row summarizing overall corporate financials.* **Relational Concept:** Derived tables (inline views), `CASE` expressions inside aggregates, and scalar joins.

Task 10: Find Movies Screened in All 8 Multiplex Theaters

Examiner Prompt: "Write a query to find movies that have at least one showtime scheduled in every single theater in the circuit (Relational Division concept)." **SQL Query:**

```
  SELECT
```

```
    m.movie_id,
    m.title
  FROM MOVIE m
  JOIN SHOWTIME s ON m.movie_id = s.movie_id
  JOIN SCREEN sc ON s.screen_id = sc.screen_id
  GROUP BY m.movie_id, m.title
  HAVING COUNT(DISTINCT sc.theater_id) = (SELECT COUNT() FROM THEATER);
```

Line-by-Line Explanation:

Joins `MOVIE` through `SHOWTIME` and `SCREEN` to access `theater_id`.

`GROUP BY m.movie_id, m.title`: Groups rows by movie.

`HAVING COUNT(DISTINCT sc.theater_id) = (SELECT COUNT() FROM THEATER)`*: Compares the distinct count of theaters where the movie is scheduled to the total count of theaters in the entire circuit (8).*

Expected Result: Returns universal releases that are playing across all venues nationwide. ***Relational Concept:*** *Relational Division ($A \div B$) simulated using* `GROUP BY`*,* `COUNT(DISTINCT)`*, and* `HAVING` *with a subquery.*

Task 11: Rank Theaters by Ticket Sales Using Window Functions

Examiner Prompt: *"Write a query to rank all multiplex theaters by total ticket sales using SQL Window Functions."* ***SQL Query:***

```
  SELECT
      t.theater_id,
      t.theater_name,
      t.city,
      COALESCE(SUM(b.total_amount), 0.00) AS theater_revenue,
      RANK() OVER (ORDER BY COALESCE(SUM(b.total_amount), 0.00) DESC) AS revenue_rank,
      DENSE_RANK() OVER (ORDER BY COALESCE(SUM(b.total_amount), 0.00) DESC) AS
dense_revenue_rank
  FROM THEATER t
  LEFT JOIN SCREEN sc ON t.theater_id = sc.theater_id
  LEFT JOIN SHOWTIME s ON sc.screen_id = s.screen_id
  LEFT JOIN BOOKING b ON s.show_id = b.show_id AND b.booking_status = 'CONFIRMED'
  GROUP BY t.theater_id, t.theater_name, t.city;
```

Line-by-Line Explanation:

`RANK() OVER (ORDER BY ... DESC)`: Calculates relative rank. If two theaters tie for rank 1, the next rank is 3.

`DENSE_RANK() OVER (...)`: Calculates dense rank without skipping rank numbers on ties.

Expected Result: *Returns all 8 multiplexes with their revenue ranks (1 through 8).* **Relational Concept:** Modern SQL Window Functions (`OVER`, `RANK()`, `DENSE_RANK()`).

Task 12: Identify Theaters with No Active Showtimes

Examiner Prompt: "Write a query to find multiplex theaters that currently have zero showtimes scheduled." **SQL Query:**

```
SELECT
    t.theater_id,
    t.theater_name,
    t.city
FROM THEATER t
WHERE t.theater_id NOT IN (
    SELECT DISTINCT sc.theater_id
    FROM SCREEN sc
    JOIN SHOWTIME s ON sc.screen_id = s.screen_id
);
```

Line-by-Line Explanation:

Subquery selects all `theater_id`s that currently appear in `SCREEN` rows linked to `SHOWTIME`.

`WHERE t.theater_id NOT IN (...)` returns theaters absent from that set.

Expected Result: Returns any newly constructed or unprogrammed multiplex venues. ***Relational Concept:*** *Set membership negation using `NOT IN`.*

Task 13: Average Ticket Price by Auditorium Screen Type

Examiner Prompt: *"Write a query to calculate the average ticket base price across different screen formats (Standard, Laser IMAX, 4DX, VIP)."* ***SQL Query:***

```
SELECT
    sc.screen_type,
    COUNT(s.show_id) AS total_screenings,
    ROUND(AVG(s.base_price), 2) AS avg_base_ticket_price,
    MIN(s.base_price) AS min_price,
    MAX(s.base_price) AS max_price
FROM SCREEN sc
JOIN SHOWTIME s ON sc.screen_id = s.screen_id
GROUP BY sc.screen_type
ORDER BY avg_base_ticket_price DESC;
```

Line-by-Line Explanation:

Groups showtimes by `sc.screen_type`.

Computes average, minimum, and maximum ticket pricing per cinema format.

Expected Result: *Shows premium formats like IMAX and 4DX commanding significantly higher average prices than Standard auditoriums.* **Relational Concept:** Group-level aggregate statistics (`AVG`, `MIN`, `MAX`, `ROUND`).

Task 14: Most Popular Screening Time Slots (Morning, Afternoon, Evening, Night)

Examiner Prompt: "Write a query to categorize showtimes into time slots (Morning, Matinee, Evening, Night) and display the number of bookings made for each slot." **SQL Query:**

```
  SELECT
      CASE
          WHEN s.start_time < '12:00:00' THEN 'Morning (Before 12 PM)'
          WHEN s.start_time BETWEEN '12:00:00' AND '16:59:59' THEN 'Matinee (12 PM - 5
PM)'
          WHEN s.start_time BETWEEN '17:00:00' AND '20:59:59' THEN 'Evening (5 PM - 9
PM)'
          ELSE 'Night (After 9 PM)'
      END AS time_slot,
      COUNT(DISTINCT s.show_id) AS scheduled_shows_count,
      COUNT(b.booking_id) AS total_bookings_received
  FROM SHOWTIME s
  LEFT JOIN BOOKING b ON s.show_id = b.show_id AND b.booking_status = 'CONFIRMED'
  GROUP BY time_slot
  ORDER BY total_bookings_received DESC;
```

Line-by-Line Explanation:

Uses a `CASE` expression to dynamically bin continuous time values into discrete business categories.

Groups and aggregates by the computed `CASE` column.

Expected Result: Reveals peak cinema attendance hours (typically Evening and Night time slots). ***Relational Concept:*** *Categorical binning using `CASE` statements and grouped aggregation.*

Task 15: Find the Most Frequently Booked Movie Genre

Examiner Prompt: *"Write a query to determine which movie genre has sold the highest number of individual seat tickets."* ***SQL Query:***

```
SELECT
    m.genre,
    COUNT(bs.seat_id) AS total_tickets_sold
FROM MOVIE m
JOIN SHOWTIME s ON m.movie_id = s.movie_id
JOIN BOOKING b ON s.show_id = b.show_id
JOIN BOOKED_SEAT bs ON b.booking_id = bs.booking_id
WHERE b.booking_status = 'CONFIRMED'
GROUP BY m.genre
ORDER BY total_tickets_sold DESC
LIMIT 1;
```

Line-by-Line Explanation:
Traverses the schema from `MOVIE` down to `BOOKED_SEAT`.

Filters strictly for confirmed bookings.

Aggregates by genre, sorts descending, and applies `LIMIT 1` to isolate the top-performing category.

Expected Result: *Returns the single top genre (e.g.,* Sci-Fi *or* Action*) along with ticket volume.* **Relational Concept:** Top-N query pattern using `ORDER BY ... DESC LIMIT 1`.

Task 16: Verify Physical Seat Map Integrity

Examiner Prompt: "Write a validation query to verify that every screen's physical seat count in the `SEAT` table matches its declared `capacity` in the `SCREEN` table." **SQL Query:**

```
SELECT
    sc.screen_id,
    sc.screen_number,
    sc.screen_name,
    sc.capacity AS declared_capacity,
    COUNT(st.seat_id) AS actual_physical_seats,
    (sc.capacity - COUNT(st.seat_id)) AS discrepancy
FROM SCREEN sc
LEFT JOIN SEAT st ON sc.screen_id = st.screen_id
GROUP BY sc.screen_id, sc.screen_number, sc.screen_name, sc.capacity
HAVING discrepancy != 0;
```

Line-by-Line Explanation:
Compares the configured integer `capacity` in `SCREEN` against the actual count of rows in `SEAT`.

`HAVING discrepancy != 0`: Filters strictly for auditoriums where a discrepancy exists.

Expected Result: Returns an empty result set (0 rows), mathematically proving 100% database integrity across all 32 screens and 4,544 physical seats. * **Relational Concept:** Database audit queries, integrity verification, and `HAVING` disparity filters.

CINEVERSE — Live Demonstration Script

5–7 Minute Examiner Presentation & Walkthrough Plan Course: Relational Database Management Systems Laboratory **Institution:** Manipal University Jaipur (MUJ) **System Services:**
Admin Frontend: `http://localhost:5173`

Customer Frontend: `http://localhost:5174`

Backend API: `http://localhost:5000`

Database: MySQL 8.0 `MovieTheaterDB`

Pre-Demonstration Setup Checklist

Before the examiner calls your name:
Ensure the MySQL service is running on port 3306.

Verify all three development servers are running:

Backend: `cd backend && node src/server.js` (Listening on 5000)

Admin: `cd frontend-admin && npm run dev` (Listening on 5173)

Customer: `cd frontend-customer && npm run dev` (Listening on 5174)

Open two browser windows side by side:

Left Window: Customer Portal (`http://localhost:5174`)

Right Window: Admin Portal (`http://localhost:5173`)

Open a terminal or MySQL Workbench connected to `MovieTheaterDB` .

Step-by-Step 5–7 Minute Walkthrough

Phase 1: High-Level Architecture & Database Blueprint (Minute 0:00 – 1:00)

What to Show:
Open the project report or ER diagram file `report_assets/cineverse_er_diagram.svg` .

Point to the separation between Venue Infrastructure (`THEATER` , `SCREEN` , `SEAT` *), Temporal Scheduling (* `SHOWTIME` *), and Commercial Ledger (* `BOOKING` , `BOOKED_SEAT` , `PAYMENT` *).*

What to Say to Examiner: *"Good morning, Examiners. Today I am presenting CINEVERSE, a multi-city multiplex movie theater booking and management system. Our database,* `MovieTheaterDB` *, is designed strictly in Third Normal Form across 15 relational tables with 22 check constraints. The system manages 8 cinema complexes across 5 Indian cities, 32 screens, 4,544 physical seats, and over 1,000 scheduled screenings."* ***Database Concept Demonstrated:*** *Three-Tier Client-Server Architecture, Conceptual ER Modeling, and 3NF Normalization.*

Phase 2: Live Database Introspection (Minute 1:00 – 2:00)

What to Show: Switch to MySQL terminal or Workbench and execute a safe read query showing table counts.

SQL Query to Execute:

```
SELECT
    table_name, table_rows
FROM information_schema.tables
WHERE table_schema = 'MovieTheaterDB';
```

What to Say to Examiner: *"Before demonstrating the web applications, here is the live MySQL database running under InnoDB. Notice the exact row distribution: 8 theaters, 32 screens, 4,544 individual seats, and 13 movies. Every table enforces primary keys, foreign keys, and explicit ON DELETE CASCADE or RESTRICT rules."*
Database Concept Demonstrated: *Physical storage engine compliance, metadata querying via* `information_schema`*, and entity integrity.*

Phase 3: Theatrical Scheduling & Collision Engine (Minute 2:00 – 3:30)

What to Show:
Navigate to the Admin Portal at `http://localhost:5173`.

Log in using `admin@cinema.com` / `Admin@123`.

Navigate to **Showtimes** (`/showtimes`).

Click **Schedule Showtime**.

Select **CINEVERSE Nexus IMAX, Bengaluru**, select **Auditorium 1 (IMAX)**, select date `2026-10-27`.

Attempt to schedule a screening at an overlapping time (e.g., 18:30 when a screening is already scheduled from 18:00 to 21:00).

Observe the red conflict alert banner: *"Scheduling Conflict: Auditorium 1 already has..."*

What to Say to Examiner: "In our admin portal, showtime scheduling is protected by a mathematical collision engine. In `showtimeController.js`, before inserting into `SHOWTIME`, the system checks for overlapping screenings using the interval condition: $A_{start} < B_{end} \land A_{end} > B_{start}$, while adding a mandatory 15-minute turnaround buffer. The database prevents duplicate bookings of the same auditorium space."
Database Concept Demonstrated: Domain constraints, temporal range integrity, and application-level pre-flight validation.

Phase 4: Dynamic Seat Map & Atomic Booking Transaction (Minute 3:30 – 5:30)

What to Show:
Switch to the Customer Portal at `http://localhost:5174`.

Point out the dynamic hero section and click ***Now Showing*** *→ select a movie (e.g.,* Oppenheimer *or* The Dark Knight*).*

Click ***Book Seats*** *to enter* `/booking-flow`.

Choose ***Bengaluru*** *→ select an evening showtime.*

The interactive seat map loads: Show that available seats are selectable, tier colors are visible (Standard, Premium, VIP), and prices adjust dynamically.

Select 2 seats (e.g., Row G, Seats 7 and 8).

Add a snack item (e.g., Caramel Popcorn).

*Click **Confirm Booking & Pay**.*

The confirmation screen appears with the Booking ID and invoice summary.

What to Say to Examiner: *"Notice what just occurred under the hood: The customer frontend fetched the live seat availability by relationally subtracting booked seats from total physical seats. When I clicked Confirm, the backend initiated a multi-statement ACID transaction. It verified seat ownership, inserted into `BOOKING`, looped over the two seats inserting into the junction table `BOOKED_SEAT`, generated a `PAYMENT` receipt, and decremented the `SNACK` stock quantity with row-level locking. If any step had failed, the entire transaction would have rolled back."* ***Database Concept Demonstrated:*** *ACID Transaction boundaries (`START TRANSACTION`, `COMMIT`, `ROLLBACK`), Associative Junction Tables ($M:N$ mapping), and Server-Side Dynamic Price Calculation.*

Phase 5: Verification of the Committed Transaction in MySQL (Minute 5:30 – 6:30)

What to Show: Switch back to the MySQL terminal and run a query showing the new booking that was just created. ***SQL Query to Execute:***

```
  SELECT
      b.booking_id,
      b.total_amount,
      b.booking_status,
      c.customer_name,
      GROUP_CONCAT(st.seat_number SEPARATOR ', ') AS seats,
      p.payment_method,
      p.payment_status
  FROM BOOKING b
  JOIN CUSTOMER c ON b.customer_id = c.customer_id
  JOIN PAYMENT p ON b.booking_id = p.booking_id
  JOIN BOOKED_SEAT bs ON b.booking_id = bs.booking_id
  JOIN SEAT st ON bs.seat_id = st.seat_id
  GROUP BY b.booking_id, b.total_amount, b.booking_status, c.customer_name,
p.payment_method, p.payment_status
  ORDER BY b.booking_id DESC
  LIMIT 1;
```

What to Say to Examiner: *"Here is our newly created booking record directly in MySQL. The booking total reflects the exact server-side calculated seat tier pricing plus concessions. The payment record is tied via a 1:1 foreign key, and the junction table `BOOKED_SEAT` holds our specific seat selections."* ***Database Concept Demonstrated:*** *Immediate consistency, relational join queries, and financial reconciliation.*

Phase 6: Conclusion & Questions (Minute 6:30 – 7:00)

What to Say to Examiner: *"To conclude, CINEVERSE demonstrates that rigorous relational database design—separating infrastructure, schedules, and reservations—eliminates data anomalies and guarantees transaction*

atomicity. The complete documentation and test suites are compiled in our official project report. I am now ready for your questions."

Troubleshooting & Fallback Protocol

Scenario / Glitch	Immediate Cause	Fallback Action to Take During Demo
Customer page shows black / blank screen	Stale Vite HMR cache in browser tab	Press **`Ctrl + Shift + R`** or **`Ctrl + F5`** immediately to hard reload the fresh bundle.
Admin login fails with "Invalid credentials"	Incorrect email or password entered	Use verified demo credentials: `email: admin@cinema.com`, `password: Admin@123`.
MySQL returns "Access Denied for user root"	Password mismatch in `.env`	Check `backend/.env`. If local MySQL uses a password, ensure `DB_PASSWORD` matches.
Port 5000 is already in use	Stale background Node process	In PowerShell run: `Get-Process node \` | Stop-Process -Force and restart `node src/server.js`.
Examiner asks to show a table that has 0 rows	Non-seeded edge case	Explain that all 15 tables are populated, and run `SELECT COUNT(*) FROM <table>` to prove data exists.

CINEVERSE — Last-Minute Rapid-Fire Revision & Cheat Sheet

High-Yield Viva Revision Guide (Read 10 Minutes Before Viva) Course: Relational Database Management Systems Laboratory (CSE Department) **Institution:** Manipal University Jaipur (MUJ)

Part 1: Project Facts & Credentials Cheat Sheet

Parameter	Exact Value	Where Verified
Database Name	`MovieTheaterDB`	MySQL Server / `backend/.env`
Storage Engine	InnoDB (ACID Compliant)	`SHOW TABLE STATUS`
Total Base Tables	15 Tables	`information_schema.tables`

Parameter	Exact Value	Where Verified
Total Database Records	5,699 Rows	Live Database Audit
Total Theaters	8 Multiplex Complexes	`THEATER` table (Bengaluru, Mumbai, Delhi, Hyd, Chennai)
Total Screens	32 Auditoriums	`SCREEN` table (4 screens per theater)
Total Physical Seats	4,544 Seats	`SEAT` table (A1..K16 per screen)
Total Movies	13 Theatrical Titles	`MOVIE` table
Total Scheduled Shows	1,008 Screenings	`SHOWTIME` table
Backend API URL / Port	`http://localhost:5000`	Node.js v20 / Express 4.19
Admin Frontend Port	`http://localhost:5173`	React 18 / Vite 5 (`frontend-admin`)
Customer Frontend Port	`http://localhost:5174`	React 18 / Vite 5 (`frontend-customer`)
Admin Login Email	`admin@cinema.com`	`ADMIN_STAFF` table
Admin Login Password	`Admin@123`	Bcrypt Hash / Lab Demo Fallback
Database Driver	`mysql2/promise` (Pool: 10)	`backend/src/config/db.js`
Security Mechanism	JWT (8h expiry) + Bcrypt (10 salt rounds)	`backend/src/middleware/auth.js`

Part 2: 30 Rapid-Fire Viva Q&As

1. What is the database name?

`MovieTheaterDB`.

2. What storage engine does MySQL use, and why?

InnoDB, because it supports ACID transactions, foreign keys, and row-level locking.

3. How many tables are in your database?

Exactly 15 base tables.

4. Name the two junction tables in your schema.

`BOOKED_SEAT` (resolves Booking ↔ Seat) and `SNACK_ORDER_ITEM` (resolves Snack Order ↔ Snack).

5. What are the composite primary keys in your project?

`BOOKED_SEAT(booking_id, seat_id)` and `SNACK_ORDER_ITEM(snack_order_id, snack_id)`.

6. What normal form does the database satisfy?

Third Normal Form (3NF) across all tables, with single-attribute PK tables satisfying BCNF.

7. Why isn't seat availability a boolean column in the SEAT table?

Because a physical seat is available or booked relative to a specific *showtime*, not permanently.

8. How is live seat availability calculated?

Available Seats = Total Screen Seats $-$ Booked Seats for that specific `show_id` in `BOOKED_SEAT`.

9. What is the mathematical condition for scheduling conflict detection?

$A_{start} < B_{end} \land A_{end} > B_{start}$ on the same screen and date.

10. How much turnaround buffer is added between movie screenings?

15 minutes (`turnaroundMins = 15`), automatically calculated in `showtimeController.js`.

11. What happens if an admin tries to delete a movie with active showtimes?

MySQL blocks the deletion with an Error 1451 foreign key constraint violation (`ON DELETE RESTRICT`).

12. What happens to child seats if a screen is deleted?

All physical seats belonging to that screen are automatically cascade-deleted (`ON DELETE CASCADE`).

13. What data type is used for financial amounts?

`DECIMAL(10,2)` to prevent binary floating-point roundoff errors.

14. What data type is used for movie ratings?

`DECIMAL(3,1)` constrained between 0.0 and 10.0 via check constraint `chk_movie_rating`.

15. How many check constraints are active in your database?

22 active CHECK constraints.

16. What happens when a booking is cancelled?

`BOOKING.booking_status` is updated to `'CANCELLED'` and `PAYMENT.payment_status` is set to `'REFUNDED'`.

17. Why aren't rows deleted from BOOKED_SEAT upon cancellation?

To preserve historical transaction audit trails while immediately releasing seat availability relationally.

18. How do you prevent SQL injection in your Node.js backend?

By strictly using parameterized queries (`?` placeholders) executed via `pool.execute()`.

19. What is a 1:1 relationship in your schema? Give two examples.

`CUSTOMER` ↔ `ACCOUNT` (enforced by unique `customer_id`) and `BOOKING` ↔ `PAYMENT` (enforced by unique `booking_id`).

20. Does your database have Triggers or Stored Procedures?

No; validations and transaction boundaries are handled in Express controllers for portability and testability.

21. What is the difference between WHERE and HAVING?

`WHERE` filters individual rows *before* grouping; `HAVING` filters aggregated groups *after* `GROUP BY`.

22. What is an intentional denormalization in your schema?

`BOOKED_SEAT.ticket_price`, which records the historical price snapshot at the moment of purchase.

23. What are the four levels of transaction isolation?

Read Uncommitted, Read Committed, Repeatable Read (MySQL default), and Serializable.

24. What does `SELECT ... FOR UPDATE` do?

Acquires exclusive row-level locks on read rows, blocking concurrent transactions until commit or rollback.

25. Where is `FOR UPDATE` used in CINEVERSE?

In `bookingController.js` for deducting `SNACK` inventory and locking the `BOOKING` row during cancellation.

26. How are seat categories priced?

Standard is base price, Premium is $+20\%$ (`base_price * 1.2`), *and VIP Recliner is $+40\%$ (* `base_price * 1.4`).

27. How are admin staff passwords protected?

Salted and hashed using `bcryptjs` with 10 salt rounds (`$2a$10$...`).

28. What token standard is used for session authentication?

JSON Web Tokens (JWT) signed with `JWT_SECRET` and expiring in 8 hours.

29. What is a candidate key? Give an example from CINEVERSE.

Any minimal superkey. In `CUSTOMER`, both `customer_id` and `email` are candidate keys.

30. How many automated tests were executed, and what was the pass rate?

37 integration tests executed with a 100% pass rate (37 passed, 0 failed).

Part 3: Golden Rules for Defending Your Viva

Be Honest About Concurrency: If asked about race conditions on seat booking, acknowledge that adding `SELECT ... FOR UPDATE` or an engine-level unique key on `(show_id, seat_id)` is the recommended production enhancement.

Never Guess Table or Column Names: Keep the cheat sheet above in mind: `SHOWTIME` uses `show_id`, `SCREEN` uses `screen_id`, and `BOOKED_SEAT` has composite PK `(booking_id, seat_id)`.

Anchor on Relational Theory: When asked why you designed a table a certain way, always use theoretical terms: *"To eliminate transitive dependencies (3NF)"*, *"To resolve Many-to-Many cardinality"*, or *"To prevent update anomalies"*.